

UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

PEDRO F. LEÃO

LUCAS P. FLORES

**DESENVOLVIMENTO DE UM SISTEMA DE *HIGH-FREQUENCY TRADING* DE
BAIXO CUSTO UTILIZANDO HLS EM FPGA**

CURITIBA

2026

**PEDRO F. LEÃO
LUCAS P. FLORES**

**DESENVOLVIMENTO DE UM SISTEMA DE *HIGH-FREQUENCY TRADING* DE
BAIXO CUSTO UTILIZANDO HLS EM FPGA**

**Development of a Low-Cost High-Frequency Trading System using HLS on
FPGA**

Trabalho de Conclusão de Curso de Graduação
apresentado como requisito para obtenção
do título de Bacharel em Engenharia de
Computação do Curso de Engenharia de
Computação da Universidade Tecnológica
Federal do Paraná.

Orientador: Prof. Carlos Raimundo Erig Lima

**CURITIBA
2026**



[4.0 Internacional](https://creativecommons.org/licenses/by/4.0/)

Esta licença permite compartilhamento, remixe, adaptação e criação a partir do trabalho, mesmo para fins comerciais, desde que sejam atribuídos créditos ao(s) autor(es). Conteúdos elaborados por terceiros, citados e referenciados nesta obra não são cobertos pela licença.

**PEDRO F. LEÃO
LUCAS P. FLORES**

**DESENVOLVIMENTO DE UM SISTEMA DE *HIGH-FREQUENCY TRADING* DE
BAIXO CUSTO UTILIZANDO HLS EM FPGA**

Trabalho de Conclusão de Curso de Graduação
apresentado como requisito para obtenção
do título de Bacharel em Engenharia de
Computação do Curso de Engenharia de
Computação da Universidade Tecnológica
Federal do Paraná.

Data de aprovação:

Prof. Carlos Raimundo Erig Lima
Orientador
Universidade Tecnológica Federal do Paraná, Campus Curitiba

Prof. Daniel Fernando Pigatto
Membro da banca
Universidade Tecnológica Federal do Paraná, Campus Curitiba

Prof. Elder Oroski
Membro da banca
Universidade Tecnológica Federal do Paraná, Campus Curitiba

**CURITIBA
2026**

Aos familiares, professores e colegas que
sustentaram a trajetória acadêmica que tornou
este trabalho possível.

Agradecimentos

Agradecemos ao Prof. Carlos Raimundo Erig Lima pela orientação, pelas discussões técnicas e pela confiança no desenvolvimento de um tema que combina arquitetura de computadores, sistemas embarcados e aplicações financeiras de baixa latência.

Agradecemos à Universidade Tecnológica Federal do Paraná, Campus Curitiba, pela formação acadêmica e pelo ambiente de aprendizagem que permitiu integrar conhecimentos de programação, eletrônica digital, redes de computadores, síntese de hardware e metodologia científica.

Agradecemos também aos colegas, familiares e amigos que contribuíram com apoio, revisão, testes, questionamentos e incentivo durante a elaboração deste trabalho.

Em sistemas de baixa latência, a previsibilidade não é apenas uma propriedade de desempenho: é parte da própria corretude operacional.

Resumo

Este trabalho apresenta o desenvolvimento de uma plataforma acadêmica de baixo custo para criação, geração e validação de estratégias de *High-Frequency Trading* (HFT) com uso de *High-Level Synthesis* (HLS) por meio do MATLAB HDL Coder. O objetivo central é permitir que a infraestrutura de comunicação, *Order Book* (livro de ofertas), integração ARM–FPGA e validação permaneça previamente preparada, de modo que o desenvolvedor concentre sua intervenção na escrita de uma função de estratégia em MATLAB compatível com síntese. A plataforma utiliza a Intel/Terasic DE10-Nano, composta por processador ARM no HPS e lógica programável Cyclone V. No fluxo proposto, um gerador publica mensagens sintéticas codificadas em FAST por TCP; um receptor em C++ decodifica os dados, normaliza símbolos, lados, preços e quantidades e envia quadros de 256 bits por MMIO; no FPGA, um *Order Book* simplificado calcula melhor compra, melhor venda, *spread* e desbalanceamento; por fim, a estratégia escrita em MATLAB é convertida para VHDL pelo HDL Coder, encapsulada por um wrapper RTL e integrada ao motor de decisão. A metodologia envolve definição do contrato ARM–FPGA, isolamento da estratégia como bloco substituível, geração automatizada de HDL, validação por testes C++, simulações VHDL com GHDL e preparação de métricas de latência, vazão, recursos FPGA e esforço de desenvolvimento. Os resultados demonstram a viabilidade funcional do fluxo como plataforma de co-projeto hardware/software e como ambiente para experimentar estratégias de trading geradas a partir de MATLAB, embora o sistema ainda não constitua uma plataforma de negociação real nem realize envio de ordens a bolsas.

Palavras-chave: fpga; *high-frequency trading*; síntese de alto nível; order book; sistemas embarcados.

Abstract

This work presents the development of a low-cost academic platform for creating, generating and validating High-Frequency Trading (HFT) strategies using High-Level Synthesis (HLS) through MATLAB HDL Coder. The main goal is to keep the communication infrastructure, order book, ARM–FPGA integration and validation flow previously prepared, so that the developer can focus on writing a synthesizable MATLAB strategy function. The platform targets the Intel/Terasic DE10-Nano, which combines an ARM processor in the HPS and Cyclone V programmable logic. In the proposed flow, a data-feed generator publishes synthetic FAST-encoded messages over TCP; a C++ receiver decodes the data, normalizes symbols, sides, prices and quantities, and sends 256-bit frames through MMIO; inside the FPGA, a simplified order book computes best bid, best ask, spread and imbalance; finally, the MATLAB strategy is converted to VHDL by HDL Coder, wrapped by RTL logic and integrated into the decision engine. The methodology covers the ARM–FPGA communication contract, isolation of the strategy as a replaceable block, automated HDL generation, C++ testing, VHDL simulation with GHDL and preparation of metrics for latency, throughput, FPGA resource usage and development effort. The results demonstrate the functional feasibility of the flow as a hardware/software co-design testbed and as an environment for experimenting with MATLAB-generated trading strategies, although the system is not yet a real trading platform and does not send orders to exchanges.

Keywords: fpga; high-frequency trading; high-level synthesis; order book; embedded systems.

Declaração de Uso de Inteligência Artificial Generativa

Durante a elaboração deste trabalho, foram utilizadas as seguintes ferramentas de Inteligência Artificial Generativa:

- **Claude Sonnet 4.6** (Anthropic): utilizado para auxílio na revisão e escrita do texto acadêmico, geração e refatoração de scripts de código (C++, VHDL, MATLAB e scripts de automação de build), e geração de casos de teste para os módulos de software e hardware.
- **GPT-5.4** (OpenAI): utilizado para auxílio na revisão e escrita do texto acadêmico, geração e refatoração de scripts de código, e geração de casos de teste.

O(s) autor(es) declara(m) que todo o conteúdo gerado pelas ferramentas acima — incluindo texto, código-fonte e casos de teste — foi integralmente revisado e validado, assumindo total responsabilidade pela precisão, integridade ética e originalidade do resultado final.

Lista de Figuras

Figura 1 – Placa DE10-Nano utilizada no protótipo	19
Figura 2 – Arquitetura conceitual do Order Book simplificado	26
Figura 3 – Arquitetura geral da plataforma proposta	42
Figura 4 – Diagrama de blocos do sistema	43
Figura 5 – Fluxo sugerido de criação de estratégia por MATLAB HDL Coder	43
Figura 6 – Ligação entre HPS e o bloco <i>HFT Trade Engine Avalon-MM</i> no Platform Designer	46
Figura 7 – Fluxo de dados do protótipo	50
Figura 8 – Pipeline lógico de atualização do livro	52

Lista de Gráficos

Lista de Tabelas

Tabela 1 – Quadro de evento enviado do ARM para o FPGA	40
Tabela 2 – Quadro de resposta enviado do FPGA para o ARM	41
Tabela 3 – Resumo da execução funcional na DE10-Nano	53
Tabela 4 – Métricas de desempenho do benchmark na DE10-Nano	56
Tabela 5 – Latência lógica vs. latência fim a fim na DE10-Nano	58
Tabela 6 – Estimativa de energia por decisão por substrato	60
Tabela 7 – Comparação de latência fim a fim entre arquiteturas de HFT	61
Tabela 8 – Registradores da interface MMIO	69
Tabela 9 – Formato do quadro ARM–FPGA	70
Tabela 10 – Variáveis de ambiente do receptor	74

Lista de Quadros

Quadro 1 – Comparação entre eixos da literatura e o protótipo	36
Quadro 2 – Matriz de validação funcional do protótipo	47
Quadro 3 – Métricas experimentais planejadas e resultados obtidos	48
Quadro 4 – Critérios de aceitação funcional	54
Quadro 5 – Riscos técnicos remanescentes e mitigação	55

Listagem de Códigos Fonte

Listagem 1 – Trecho da saída do receptor com respostas FPGA–ARM	53
Listagem 2 – Comandos de teste local	73
Listagem 3 – Geração do bloco de estratégia por HDL Coder	73
Listagem 4 – Fluxo previsto para DE10-Nano	74
Listagem 5 – Execução na placa	74

Lista de Abreviaturas e Siglas

Siglas

ABNT	Associação Brasileira de Normas Técnicas
API	Interface de Programação de Aplicações, do inglês <i>Application Programming Interface</i>
ARM	Família de arquiteturas de processadores RISC utilizada no subsistema embarcado
CPU	Unidade Central de Processamento, do inglês <i>Central Processing Unit</i>
DMA	Acesso Direto à Memória, do inglês <i>Direct Memory Access</i>
FAST	FIX Adapted for STreaming
FIX	Financial Information eXchange
FPGA	Arranjo de Portas Programáveis em Campo, do inglês <i>Field-Programmable Gate Array</i>
GHDL	Simulador de VHDL utilizado nos testes automatizados
HDL	Linguagem de Descrição de Hardware, do inglês <i>Hardware Description Language</i>
HFT	Negociação de Alta Frequência, do inglês <i>High-Frequency Trading</i>
HLS	Síntese de Alto Nível, do inglês <i>High-Level Synthesis</i>
HPS	Sistema Processador Rígido, do inglês <i>Hard Processor System</i>
IP	Propriedade Intelectual reutilizável de hardware, do inglês <i>Intellectual Property</i>
MMIO	Entrada e Saída Mapeada em Memória, do inglês <i>Memory-Mapped Input/Output</i>
PDF	Formato de Documento Portátil, do inglês <i>Portable Document Format</i>
RTL	Nível de Transferência entre Registradores, do inglês <i>Register Transfer Level</i>
SoC	Sistema em um Chip, do inglês <i>System-on-Chip</i>
TCP	Protocolo de Controle de Transmissão, do inglês <i>Transmission Control Protocol</i>
UTFPR	Universidade Tecnológica Federal do Paraná
VCD	Value Change Dump
VHDL	VHSIC Hardware Description Language

Lista de Símbolos

Letras Latinas

D	Profundidade do anel ou do <i>Order Book</i> (livro de ofertas)	
H	Ponteiro de produção de uma fila circular	
T	Ponteiro de consumo de uma fila circular	
L	Latência observada entre entrada e resposta	s
Q	Quantidade agregada em um nível de preço	
P	Preço representado em ponto fixo escalado por 10^4	
S	<i>Spread</i> entre melhor venda e melhor compra	
V	Vazão de processamento de eventos	eventos/s

Letras Gregas

Δ	Diferença entre duas grandezas, como preço ou tempo	
λ	Taxa média de chegada de eventos de mercado	eventos/s

Sobrescritos

+	Lado comprador do <i>Order Book</i>
−	Lado vendedor do <i>Order Book</i>

Subscritos

bid	Melhor oferta de compra
ask	Melhor oferta de venda
tx	Fila de transmissão ARM–FPGA
rx	Fila de recepção FPGA–ARM

Notações

$next(x)$	Próxima posição de uma fila circular, calculada com aritmética modular
$price_1e4$	Preço inteiro resultante da multiplicação do preço decimal por 10^4

Sumário

1	Introdução	18
1.1	Considerações iniciais	18
1.2	Problema de pesquisa	19
1.3	Objetivos	20
1.3.1	Objetivo geral	20
1.3.2	Objetivos específicos	20
1.4	Justificativa	21
1.5	Estrutura do trabalho	22
2	Referencial teórico	23
2.1	Negociação de alta frequência	23
2.1.1	Caminho crítico em sistemas de HFT	23
2.1.2	Determinismo e variabilidade	24
2.2	Order Book (livro de ofertas) e dados de mercado	24
2.2.1	Eventos por nível e eventos por ordem	25
2.2.2	Sinais derivados do topo do livro	25
2.3	FPGA, SoC e plataforma DE10-Nano	27
2.3.1	Comparação entre CPU e FPGA	27
2.3.2	Papel do HPS	28
2.4	Síntese de alto nível e MATLAB HDL Coder	28
2.4.1	Restrições de código sintetizável	28
2.4.2	Por que não gerar todo o sistema por HLS	29
2.5	FAST, mFAST e fluxo TCP	29
2.5.1	Template de mensagem	29
2.5.2	Limites do feed sintético	30
2.6	MMIO, Avalon-MM e filas circulares	30
2.6.1	Ordenação de publicação	30
2.6.2	Avalon-MM como camada de integração	31
2.6.3	MMIO versus DMA	31
3	Trabalhos Relacionados	33
3.1	Negociação algorítmica e baixa latência	33

3.2	Aceleração de HFT com FPGA	33
3.3	Order Book e modelos de atualização	34
3.4	Síntese de alto nível e fronteiras de projeto	35
3.5	Comparação com o protótipo desenvolvido	35
3.6	Posicionamento do trabalho	35
4	Proposta	37
4.1	Visão geral da arquitetura	37
4.2	Decisões arquiteturais	38
4.3	Fluxo de criação de estratégias com MATLAB HDL Coder	38
4.4	Modos de operação e delimitações	39
4.5	Contrato de eventos ARM–FPGA	40
4.6	Mapeamento de símbolos e geração de dados	40
4.7	Regra de estratégia	41
5	Materiais e Métodos	44
5.1	Materiais	44
5.2	Métodos	44
5.2.1	Organização da plataforma	44
5.2.2	Integração no Quartus e Platform Designer	45
5.2.3	Contrato de comunicação	45
5.2.4	Fluxo de dados de mercado	46
5.2.5	Order Book e estratégia	47
5.2.6	Validação funcional	47
5.2.7	Método proposto para métricas experimentais	48
6	Resultados	49
6.1	Plano da campanha experimental	49
6.2	Sistema obtido	50
6.3	Modelagem funcional	51
6.4	Execução funcional na DE10-Nano	51
6.5	Validação por testes	53
6.6	Discussão	54

6.7	Benchmark de desempenho	55
6.7.1	Metodologia de medição	55
6.7.2	Resultados obtidos	56
6.7.3	Determinismo e ausência de jitter	57
6.7.4	Gargalo de comunicação e latência fim a fim	58
6.7.5	Eficiência energética	59
6.7.6	Comparação com trabalhos relacionados	60
7	Conclusão	63
	REFERÊNCIAS	66
	APÊNDICE A Contrato de Comunicação ARM–FPGA	69
	A.1 Registradores	69
	A.2 Regras de filas	69
	A.3 Formato de evento	69
	A.4 Formato de resposta	70
	A.5 Checklist de alteração do contrato	70
	APÊNDICE B Comandos de Construção, Teste e Execução	73
	B.1 Validação local	73
	B.2 Geração da estratégia	73
	B.3 Geração HDL e build para DE10-Nano	73
	B.4 Variáveis de ambiente	74
	B.5 Sequência recomendada de depuração	74

1 Introdução

Mercados eletrônicos modernos são sistemas computacionais distribuídos nos quais a formação de preços depende de mensagens de oferta, cancelamento, execução e atualização de Order Book, isto é, do livro de ofertas, que percorrem redes de baixa latência. Nesse cenário, a capacidade de receber dados de mercado, atualizar estado interno e produzir uma decisão em intervalos previsíveis tornou-se um requisito técnico central para aplicações de negociação algorítmica e, em especial, para HFT. A literatura de microestrutura descreve que a redução de latência altera a dinâmica competitiva dos participantes e influencia liquidez, descoberta de preços e comportamento do Order Book (HASBROUCK; SAAR, 2013; HENDERSHOTT; JONES; MENKVELD, 2011; BROGAARD; HENDERSHOTT; RIORDAN, 2014).

Do ponto de vista de engenharia de computação, a dificuldade não está apenas em executar uma regra de negociação, mas em construir uma cadeia de processamento com baixo tempo de resposta, controle explícito de filas, representação adequada dos dados e integração entre software e hardware. CPUs de uso geral oferecem flexibilidade e ecossistema maduro, porém dependem de execução sequencial, hierarquia de memória, interrupções, escalonamento de sistema operacional e variabilidade temporal. FPGAs, por outro lado, permitem transformar partes do fluxo em circuitos dedicados, explorando paralelismo espacial e latência determinística no nível de ciclos de clock (HARRIS; HARRIS, 2013; PATTERSON; HENNESSY, 2017).

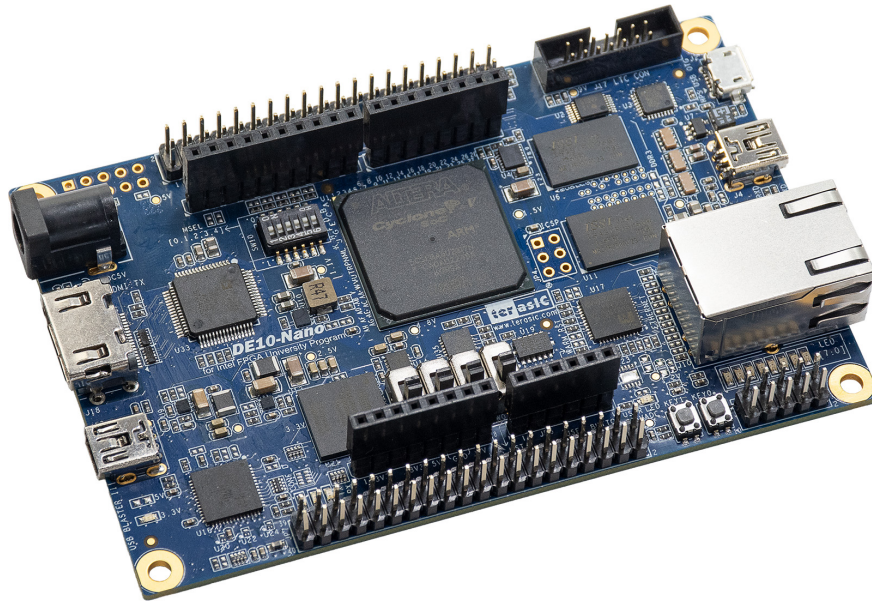
Este trabalho investiga essa alternativa por meio de uma plataforma acadêmica de baixo custo baseada na Intel/Terasic DE10-Nano, que combina processador ARM no HPS com lógica programável Cyclone V. A contribuição principal está no uso de HLS com MATLAB HDL Coder para transformar uma estratégia de trading escrita em MATLAB em um bloco VHDL integrável a uma infraestrutura previamente preparada. Nessa abordagem, a comunicação ARM–FPGA, o fluxo de dados de mercado, o Order Book simplificado, os testes e o projeto Quartus formam a base do ambiente; o ponto de variação passa a ser a estratégia, que pode ser redigida em MATLAB e regenerada para hardware.

1.1 Considerações iniciais

O termo HFT é empregado neste trabalho para representar sistemas automatizados que processam dados de mercado e emitem decisões em janelas temporais muito reduzidas. A implementação desenvolvida não se conecta a uma bolsa real, não executa ordens financeiras e não deve ser interpretada como sistema de produção. Ela é uma plataforma experimental para estudar os elementos computacionais que aparecem em uma cadeia de HFT: recepção de mensagens, normalização de eventos, estrutura de Order Book, decisão algorítmica, medição de comportamento e integração com lógica programável.

A escolha por uma plataforma de baixo custo tem relevância acadêmica. Infraestruturas profissionais de HFT utilizam enlaces especializados, placas de rede de latência ultrabaixa,

Figura 1 – Placa DE10-Nano utilizada no protótipo



Fonte: Terasic Technologies (Terasic Technologies, 2024)..

kernel bypass, co-localização e FPGAs de alto desempenho. Esses recursos são financeiramente inacessíveis para a maioria dos laboratórios de graduação. A DE10-Nano, ilustrada na Figura 1, apesar de limitada em frequência, memória e recursos lógicos quando comparada a placas industriais, oferece um ambiente suficientemente rico para estudar os princípios de integração entre ARM, FPGA, barramentos, síntese de hardware e processamento determinístico (Terasic Technologies, 2024; Intel Corporation, 2025). Esse caráter de baixo custo não é apenas qualitativo: a própria página de compra da Terasic lista a DE10-Nano em condição acadêmica por US\$190 (Terasic Technologies, 2026). Para efeito de contraste, FPGAs de classe usada em trabalhos de HFT de alta performance podem custar ordens de grandeza a mais; por exemplo, o dispositivo AMD/Xilinx XCKU115-2FLVA1517E aparece listado pela Mouser por US\$10.290,57 (Mouser Electronics, 2026). Mesmo sendo uma comparação conservadora — pois contrapõe uma placa completa a um único FPGA de alto desempenho — ela já evidencia a diferença de orçamento entre uma plataforma didática e uma solução de classe profissional.

1.2 Problema de pesquisa

O problema tratado neste TCC pode ser formulado da seguinte forma: como construir, em uma plataforma FPGA de baixo custo, uma infraestrutura que permita implementar estratégias de trading com menor esforço de descrição RTL, preservando determinismo, paralelismo em hardware e integração com um fluxo de dados de mercado? Essa pergunta combina duas dificuldades. A primeira é arquitetural: sistemas baseados apenas em CPU sofrem com processamento sequencial, overhead do sistema operacional, variabilidade de escalonamento e limitações do modelo von Neumann. A segunda é metodológica: escrever toda a lógica de de-

cisão diretamente em VHDL aumenta a barreira de entrada para testar novas estratégias. A hipótese de trabalho é que separar infraestrutura estável e estratégia variável pode reduzir essa barreira sem abrir mão da execução em FPGA.

Do ponto de vista técnico, os requisitos derivados desse problema são: baixa variabilidade temporal no trecho em hardware; representação numérica fixa para preços e quantidades; processamento de Order Book em estrutura de profundidade limitada; interface ARM–FPGA documentada; validação por simulação e execução em placa; e planejamento de métricas de latência, vazão, uso de recursos e produtividade na alteração de estratégias.

1.3 Objetivos

Esta seção apresenta o objetivo geral e os objetivos específicos do trabalho.

1.3.1 Objetivo geral

Desenvolver e documentar uma plataforma de HFT de baixo custo, baseada em ARM e FPGA, cujo objetivo principal é facilitar a criação de estratégias de trading em MATLAB e sua geração para hardware por meio do MATLAB HDL Coder, mantendo previamente preparados o fluxo de dados de mercado, o Order Book simplificado, a comunicação ARM–FPGA e a infraestrutura de validação.

1.3.2 Objetivos específicos

Os objetivos específicos são:

- estruturar um ambiente no qual a estratégia de trading seja o principal bloco substituível pelo usuário;
- definir um modelo de estratégia em MATLAB compatível com HDL Coder, utilizando tipos sintetizáveis, entradas numéricas e saída de ação;
- automatizar o fluxo de geração VHDL da estratégia a partir do código MATLAB;
- definir o contrato de comunicação entre software ARM e lógica FPGA, incluindo registradores, ponteiros, semântica de filas e formato dos quadros;
- implementar e explicar o fluxo de geração e recepção de mensagens FAST sintéticas utilizado como alimentação de mercado;
- manter, no FPGA, um Order Book por níveis de preço com profundidade fixa, ordenação por lado e cálculo de melhor compra, melhor venda, *spread* e desbalanceamento;

- integrar a estratégia gerada por MATLAB HDL Coder ao motor VHDL por meio de uma interface estável;
- validar funcionalmente os componentes por testes C++, simulações VHDL e teste de fumaça entre gerador e receptor;
- propor métricas para avaliar latência, vazão, uso de recursos FPGA e esforço necessário para alterar uma estratégia.

1.4 Justificativa

A relevância do trabalho decorre da interseção entre arquitetura de computadores, sistemas embarcados, redes e aplicações financeiras de baixa latência. Em HFT, pequenas variações de tempo de resposta podem modificar a ordem de chegada de mensagens e, consequentemente, a decisão disponível ao participante. A literatura mostra que atividades de negociação algorítmica e baixa latência afetam liquidez e descoberta de preços, ainda que os efeitos dependam do mercado, do período e da estratégia adotada (HENDERSHOTT; JONES; MENKVELD, 2011; BROGAARD; HENDERSHOTT; RIORDAN, 2014; HASBROUCK; SAAR, 2013). Portanto, estudar a infraestrutura computacional subjacente é relevante para compreender tanto ganhos de desempenho quanto riscos técnicos.

Do ponto de vista pedagógico, o trabalho força a integração de conteúdos normalmente vistos de forma separada. O receptor C++ exige programação de sistemas, sockets TCP, manipulação binária e interface com bibliotecas externas. O bloco FPGA exige VHDL, filas circulares, protocolos de barramento, controle de *handshake* e testbenches. A etapa MATLAB HDL Coder introduz HLS e as restrições de escrever código algorítmico sintetizável.

A escolha por FPGA é justificada pelo tipo de carga de trabalho. Atualizações de Order Book, empacotamento de quadros, cálculo de *spread*, comparação de limiares e geração de ação são operações regulares, com largura fixa e baixa necessidade de memória dinâmica. Essas características são favoráveis à implementação em hardware dedicado. Em contrapartida, decodificação de protocolo, reconexão TCP, mensagens de erro e interação com sistema operacional permanecem no ARM, onde a flexibilidade de software é mais apropriada. Esse co-projeto evita uma solução puramente acadêmica desconectada das restrições práticas de depuração e integração.

Além da motivação técnica, há impacto educacional e econômico. Educacionalmente, o trabalho aproxima sistemas embarcados, arquitetura de computadores, mercado financeiro, HLS e validação experimental em um único projeto. Economicamente, a escolha por uma DE10-Nano e por ferramentas acadêmicas permite avaliar uma alternativa de baixo custo frente a soluções comerciais especializadas. Cientificamente, o projeto contribui com uma análise prática de como ferramentas modernas de HLS podem ser aplicadas a um domínio sensível a latência, mas sem exigir que cada nova estratégia seja escrita manualmente em RTL.

1.5 Estrutura do trabalho

O Capítulo 2 apresenta o referencial teórico utilizado para compreender HFT, Order Book, FPGA, HLS, MATLAB HDL Coder, FAST, MMIO, Avalon-MM, filas circulares e métricas de desempenho. O Capítulo 3 discute trabalhos relacionados em negociação algorítmica, aceleração em FPGA, HLS e processamento de Order Books. O capítulo de proposta descreve a plataforma desenvolvida, suas fronteiras, o fluxo de geração de estratégias e seus contratos de dados. O Capítulo 5 detalha materiais, ferramentas, métodos de desenvolvimento e planejamento de métricas. O Capítulo 6 apresenta o sistema obtido, a modelagem, a implementação e os espaços reservados para resultados quantitativos. Por fim, o Capítulo 7 sintetiza as contribuições, limitações e perspectivas futuras.

2 Referencial teórico

Este capítulo apresenta os conceitos necessários para compreender o trabalho. A organização acompanha os tópicos que sustentam a proposta: mercado e Order Book, processamento de mensagens, arquitetura ARM–FPGA, comunicação por MMIO, projeto de hardware em VHDL e HLS, e critérios de desempenho.

2.1 Negociação de alta frequência

HFT é uma forma de negociação algorítmica caracterizada por tomada de decisão automatizada, alto volume de mensagens, horizonte temporal curto e forte dependência de latência. Em vez de avaliar fundamentos econômicos em períodos longos, sistemas desse tipo exploram mudanças rápidas no Order Book, desequilíbrios de fluxo, arbitragem entre instrumentos e oportunidades temporárias de liquidez. A literatura associa HFT à infraestrutura de baixa latência, à automação de estratégias e à necessidade de responder a eventos de mercado antes que a informação se torne obsoleta (ALDRIDGE, 2013; HASBROUCK; SAAR, 2013).

Em uma implementação real, o caminho crítico inclui recepção de pacotes, decodificação de protocolo, normalização de mensagens, atualização de estado de mercado, execução da estratégia, controle de risco, roteamento de ordens e confirmação. Este trabalho cobre apenas uma parte desse caminho: dados sintéticos de mercado, Order Book simplificado, decisão elementar e retorno de resposta ao software. A delimitação é importante porque evita afirmar que o protótipo realiza HFT operacional; ele reproduz elementos computacionais relevantes para estudar latência e arquitetura.

As vantagens de uma abordagem em hardware são a previsibilidade temporal e a exploração de paralelismo. Comparações, somas, deslocamentos em arranjos pequenos e empacotamento de palavras podem ocorrer em circuitos dedicados, sem depender de escalonamento de CPU. As limitações são menor flexibilidade, maior custo de depuração, necessidade de definir larguras fixas e dificuldade para lidar com protocolos complexos. Por isso, o projeto utiliza hardware para o núcleo regular do problema e software para a borda flexível.

2.1.1 Caminho crítico em sistemas de HFT

Em sistemas de negociação de baixa latência, o caminho crítico é a sequência mínima de operações entre a chegada de uma informação de mercado e a emissão de uma resposta. Mesmo quando a estratégia financeira é simples, a infraestrutura precisa executar diversas etapas: receber bytes da rede, identificar fronteiras de mensagem, decodificar campos, atualizar estado, avaliar condições, aplicar filtros de segurança, serializar uma resposta e transmiti-la. Cada etapa adiciona latência e, principalmente, variabilidade. Essa variabilidade é relevante

porque dois eventos com latências médias semelhantes podem ter comportamentos muito diferentes em cauda, isto é, nos piores casos.

Em uma pilha tradicional baseada em CPU, o caminho crítico atravessa camadas de software geral: driver de rede, pilha TCP/IP ou UDP/IP, cópias de buffer, chamadas de sistema, escalonador, cache, predição de desvio e sincronização entre threads. Técnicas profissionais como *kernel bypass*, afinidade de CPU e *busy polling* reduzem parte desse custo, mas aumentam complexidade e dependência de hardware específico. A alternativa com FPGA desloca parte do caminho para uma lógica síncrona, na qual comparações e atualizações podem ocorrer em ciclos previsíveis.

A abordagem estudada não elimina a pilha de rede, pois a recepção TCP e a decodificação FAST continuam no ARM. Entretanto, ela isola o trecho de atualização de livro e decisão em hardware. Assim, o caminho crítico analisado é menor que o de uma plataforma profissional, mas suficiente para investigar os fundamentos de co-projeto.

2.1.2 Determinismo e variabilidade

Determinismo, neste trabalho, não significa que o sistema sempre responderá no mesmo tempo absoluto em execução Linux completa. Significa que o trecho descrito em hardware possui transições controladas por clock e quantidade limitada de estados. Uma atualização de livro com profundidade fixa não depende de alocação dinâmica, coleta de lixo, paginação de memória ou contenção de locks. O custo fica associado à profundidade configurada e ao caminho combinacional/síncrono gerado.

Essa propriedade é especialmente importante em aplicações de Order Book. A cada evento, o sistema deve preservar invariantes: melhor compra abaixo da melhor venda, lados corretamente ordenados, quantidade nula removendo nível e resposta correlacionada à sequência recebida. Se a infraestrutura é não determinística, torna-se difícil diferenciar erro funcional de atraso ocasional. Ao manter a lógica central em VHDL e validá-la por testbench, o projeto reduz esse espaço de ambiguidade.

2.2 Order Book (livro de ofertas) e dados de mercado

O Order Book limitado, ou *limit order book*, é a estrutura que organiza intenções de compra e venda por preço e quantidade. Em português, esse termo corresponde ao livro de ofertas. O lado comprador contém ofertas de compra, com melhor compra no maior preço disponível; o lado vendedor contém ofertas de venda, com melhor venda no menor preço disponível. A diferença entre melhor venda e melhor compra é o *spread*. A quantidade disponível nos melhores níveis e seu desbalanceamento podem ser usados como sinais simples de pressão compradora ou vendedora (GOULD *et al.*, 2013; CONT; STOIKOV; TALREJA, 2010).

Um livro completo em bolsa pode conter ordens individuais, prioridades por tempo, cancelamentos parciais, execuções e múltiplos tipos de evento. O protótipo adota uma abstração por nível de preço: cada evento informa símbolo, lado, preço e quantidade agregada. O FPGA mantém, para cada símbolo, vetores de preço e quantidade com profundidade fixa. No lado comprador, os preços são ordenados de forma decrescente; no lado vendedor, de forma crescente. Atualizações existentes alteram quantidade; quantidades nulas ou eventos de remoção retiram o nível; novas entradas deslocam níveis inferiores até o limite da profundidade.

A principal vantagem dessa modelagem é a determinização do custo computacional. Com número fixo de símbolos e profundidade fixa, o circuito pode ser descrito com laços de limite conhecido e memória estática. A principal limitação é a perda de fidelidade em relação a um Order Book real, pois não há prioridade por ordem individual, identificador real de ordem, execução parcial ou múltiplas bolsas. Ainda assim, a abstração é adequada para avaliar o fluxo de dados e a integração FPGA.

2.2.1 Eventos por nível e eventos por ordem

Há duas formas comuns de representar atualizações de livro. Na representação por ordem, cada mensagem descreve uma ordem individual, identificada por um código, com preço, quantidade, lado e estado. Cancelamentos e modificações referem-se a esse identificador. Essa abordagem é mais fiel a mercados reais, mas exige estruturas de busca por identificador, controle de prioridade temporal e tratamento de execuções parciais. Em hardware, isso tende a exigir memórias associativas, tabelas de dispersão ou estruturas indexadas, aumentando o custo de síntese.

Na representação por nível, usada neste protótipo, cada mensagem atualiza diretamente uma quantidade agregada em determinado preço e lado. A operação é mais simples: encontrar o preço, substituir a quantidade, inserir o nível ou removê-lo. O custo computacional passa a depender da profundidade do livro, não do número total de ordens. A consequência é que o sistema perde a capacidade de reconstruir a fila de prioridade interna de cada preço. Para a plataforma proposta, essa simplificação é vantajosa porque permite discutir ordenação, *spread* e desbalanceamento sem implementar uma bolsa completa.

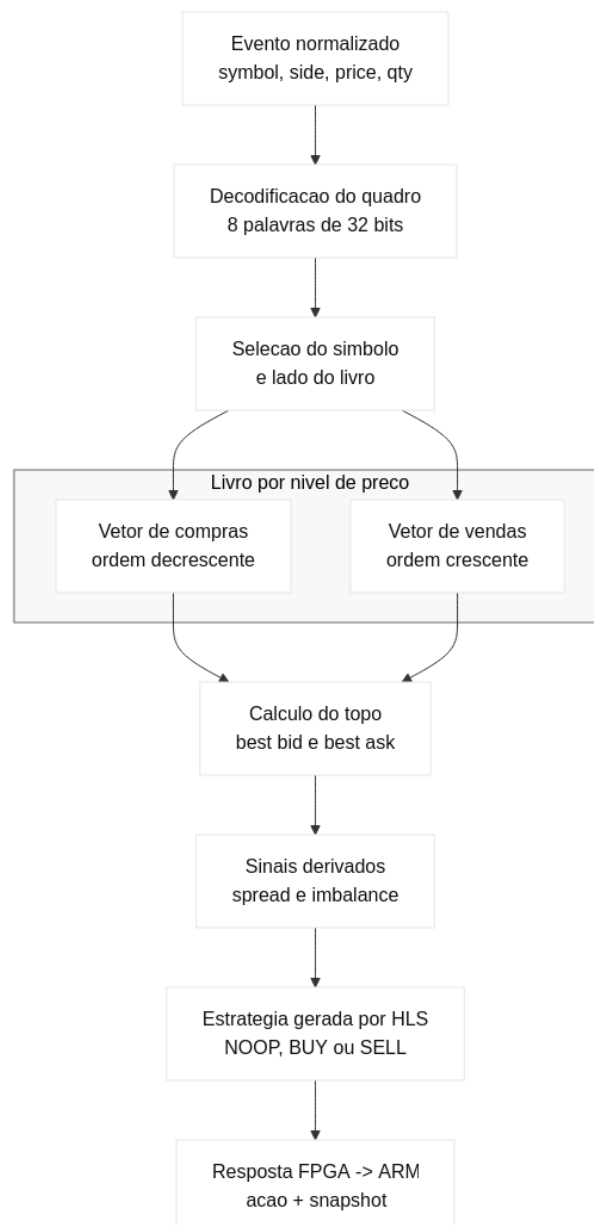
2.2.2 Sinais derivados do topo do livro

O topo do livro é formado pela melhor oferta de compra e pela melhor oferta de venda. A partir dele, o protótipo calcula o *spread* e o desbalanceamento de quantidade. O *spread* é calculado apenas quando ambos os lados existem e a melhor venda é maior que a melhor compra. O desbalanceamento é calculado como a diferença assinada entre quantidade no melhor nível

de compra e quantidade no melhor nível de venda. Valores positivos indicam maior quantidade compradora no topo; valores negativos indicam maior quantidade vendedora.

Esses sinais são simplificações de métricas usadas em microestrutura de mercado. Em sistemas reais, também poderiam ser considerados profundidade acumulada em múltiplos níveis, taxa de cancelamento, agressões recentes, tempo desde a última atualização, volatilidade local e posição atual da estratégia. O protótipo usa apenas sinais de topo porque eles cabem naturalmente em um quadro de resposta de oito palavras e são fáceis de validar por simulação.

Figura 2 – Arquitetura conceitual do Order Book simplificado



Fonte: Elaborado pelos autores a partir de diagrama Mermaid..

2.3 FPGA, SoC e plataforma DE10-Nano

Uma FPGA é um circuito integrado reconfigurável formado por blocos lógicos, interconexões programáveis, registradores, memórias internas, multiplicadores e interfaces especializadas. Diferentemente de uma CPU, que executa instruções sequenciais sobre uma unidade funcional compartilhada, a FPGA implementa diretamente o caminho de dados e o controle da aplicação. Essa característica permite paralelismo no nível de operações e previsibilidade em ciclos de clock, desde que a aplicação seja descrita com recursos e temporização adequados (HARRIS; HARRIS, 2013).

A família Intel Cyclone V SoC combina lógica FPGA com um HPS baseado em ARM Cortex-A9, controladores de memória e pontes de comunicação entre processador e malha programável (Intel Corporation, 2025). A DE10-Nano utiliza esse SoC em uma placa educacional, com recursos suficientes para protótipos de sistemas embarcados e aceleração por hardware (Terasic Technologies, 2024). Para este trabalho, a característica mais relevante não é a quantidade absoluta de recursos lógicos, mas a existência de uma ponte HPS–FPGA acessível a partir do Linux embarcado.

O uso de um SoC FPGA traz um compromisso técnico. O ARM executa Linux, C++ e bibliotecas de protocolo com maior facilidade; a lógica programável executa operações regulares com latência previsível. A comunicação entre ambos, entretanto, torna-se parte crítica do projeto. Um acelerador muito rápido perde utilidade se o custo de transferência for alto, se o contrato de memória for ambíguo ou se houver divergência entre software, documentação e RTL.

2.3.1 Comparação entre CPU e FPGA

A CPU é otimizada para flexibilidade. Ela executa um fluxo de instruções, possui caches, unidades vetoriais, predição de desvios e suporte maduro a sistemas operacionais. Essa flexibilidade é ideal para protocolos complexos, tratamento de erro, alocação de objetos, logs e integração com bibliotecas. Contudo, a CPU compartilha unidades funcionais entre muitas tarefas e depende de uma microarquitetura projetada para desempenho médio elevado em cargas variadas.

A FPGA é otimizada para especialização. Um comparador, um somador e um registrador podem existir fisicamente como parte do circuito da aplicação. Se o projeto exige várias comparações independentes, elas podem ocorrer simultaneamente, desde que haja recursos lógicos suficientes. Essa especialização reduz flexibilidade: alterar a estrutura exige nova síntese e programação do bitstream. No protótipo, essa troca é aceitável porque o Order Book tem profundidade pequena, formato fixo e operações bem definidas.

2.3.2 Papel do HPS

O HPS da DE10-Nano é o ponto de contato entre o mundo de software e a lógica reconfigurável. Ele executa Linux, sockets, bibliotecas e acesso a memória mapeada. O FPGA, por sua vez, opera com sinais, registradores e barramentos. A fronteira entre os dois domínios é, portanto, uma tradução de abstrações: mensagens e estruturas de software tornam-se palavras de largura fixa e sinais de controle em hardware.

Essa fronteira explica por que o projeto investe tanto no contrato MMIO. Se o software assume uma largura de quadro e o hardware sintetiza outra, o erro não aparece como exceção de alto nível; aparece como palavra trocada, ponteiro inconsistente ou resposta incorreta. A disciplina de documentação e teste é o mecanismo usado para manter o HPS e o FPGA sincronizados.

2.4 Síntese de alto nível e MATLAB HDL Coder

HLS é o processo de transformar uma descrição algorítmica de alto nível em uma implementação de hardware em RTL. Em vez de escrever manualmente todos os registradores, estados e sinais de controle, o projetista descreve operações em uma linguagem como C, C++ ou MATLAB, e a ferramenta gera VHDL ou Verilog. O MATLAB HDL Coder permite gerar código HDL a partir de funções MATLAB e modelos Simulink, desde que o código respeite restrições de sintetizabilidade, tipos fixos e tamanhos conhecidos (The MathWorks, Inc., 2026b).

Neste trabalho, a síntese de alto nível é usada de forma conservadora. A infraestrutura de comunicação e o Order Book são mantidos em VHDL manual, enquanto a regra decisória fica em MATLAB e é convertida para hardware pelo HDL Coder. Essa separação reduz o risco de depender de código gerado em toda a arquitetura e concentra a flexibilidade no ponto que mais deve variar: a estratégia.

A vantagem dessa abordagem é permitir que regras algorítmicas sejam alteradas em MATLAB, ambiente familiar para modelagem matemática. A limitação é que nem todo MATLAB é sintetizável: alocação dinâmica, strings, estruturas variáveis, bibliotecas não suportadas e ponto flutuante irrestrito podem inviabilizar a geração. O projeto evita esses problemas com tipos inteiros, limiares constantes e ponto fixo escalado para representar preços (The MathWorks, Inc., 2026a).

2.4.1 Restrições de código sintetizável

Uma função MATLAB convencional pode usar vetores de tamanho variável, estruturas heterogêneas, strings, objetos, arquivos e operações dependentes do ambiente de execução. Uma função sintetizável precisa ser muito mais restrita. As entradas e saídas devem ter tipos e

larguras conhecidas; laços precisam ter limites definidos; a aritmética deve ser representável em hardware; e o comportamento não pode depender de recursos que só existem no interpretador ou no sistema operacional.

Neste trabalho, a estratégia recebe apenas sinais já calculados do Order Book: preços, quantidades, *spread* e desbalanceamento. Ela não percorre o livro completo nem depende de estruturas dinâmicas. Essa escolha mantém o bloco gerado pequeno e previsível.

2.4.2 Por que não gerar todo o sistema por HLS

Embora HLS seja útil, gerar toda a ponte ARM–FPGA por MATLAB HDL Coder seria uma decisão arriscada. Barramentos, registradores, filas e handshakes dependem de contratos precisos com software e Platform Designer. Esses elementos são mais transparentes quando escritos manualmente em VHDL. Além disso, a depuração de um erro de ponte gerada automaticamente pode ser mais difícil do que a depuração de um bloco RTL pequeno e explícito.

Por isso, a infraestrutura de comunicação e armazenamento é mantida em RTL manual, enquanto a política de decisão, que tende a mudar mais durante experimentos, fica em MATLAB.

2.5 FAST, mFAST e fluxo TCP

FAST, sigla de *FIX Adapted for STreaming*, é um padrão de codificação binária orientado a reduzir o tamanho de mensagens financeiras ao explorar templates, operadores de cópia, incremento e delta (FIX Trading Community, 2006). Em mercados reais, protocolos desse tipo reduzem banda e custo de transmissão de dados de mercado. Na plataforma proposta, FAST é usado como elemento de realismo controlado: o feed sintético produz mensagens de mercado e o receptor as decodifica com apoio da biblioteca mFAST (Object Computing, Inc., 2026).

A comunicação de rede do protótipo é TCP local na porta 9001. O servidor transmite cada mensagem precedida por um comprimento em ordem de rede, permitindo que o cliente leia exatamente a quantidade de bytes necessária antes de chamar o decodificador. Essa decisão simplifica o ambiente experimental, pois evita lidar com multicast, UDP, perda de pacotes e *kernel bypass*. A limitação é que TCP e localhost não reproduzem a pilha de dados de mercado de uma bolsa real. Para um TCC focado na integração ARM–FPGA e na geração de estratégias por HLS, a simplificação é aceitável porque mantém o feed determinável e testável.

2.5.1 Template de mensagem

O template de mensagem define os campos usados no protótipo: símbolo, lado, preço, quantidade e sequência. Ele emprega operadores típicos de FAST para explorar redundância entre mensagens sucessivas. A biblioteca mFAST gera estruturas tipadas para o receptor,

reduzindo a necessidade de manipular manualmente offsets binários. A desvantagem é a dependência de uma etapa adicional de geração e build.

2.5.2 Limites do feed sintético

O feed sintético escolhe símbolos e lados pseudoaleatoriamente, usa preços base e aplica deslocamentos pequenos. Isso permite gerar um fluxo contínuo sem depender de dados proprietários. Entretanto, não há bursts realistas, leilões, mensagens de abertura, pacotes fora de ordem, perda de rede, múltiplas sessões ou sincronização de relógio. Assim, os resultados obtidos com esse feed devem ser interpretados como validação funcional, não como simulação estatística completa de mercado.

2.6 MMIO, Avalon-MM e filas circulares

MMIO é uma técnica em que registradores de dispositivos são expostos no espaço de endereçamento do processador. Leituras e escritas em endereços específicos se transformam em transações de barramento. No Linux embarcado da DE10-Nano, o receptor pode mapear a região física associada à ponte HPS–FPGA e usar esse espaço para trocar eventos com o hardware.

No lado FPGA, a integração utiliza Avalon-MM, protocolo de memória mapeada empregado pelo Platform Designer da Intel para conectar componentes no modelo *master/slave* em sistemas SoC (Intel Corporation, 2024a; Intel Corporation, 2024b). O wrapper Avalon-MM adapta as transações do HPS, no papel de *master*, para a interface interna do motor HFT, exposta como periférico do tipo *slave*. A presença de uma política de *timeout* auxilia a depuração durante reset ou integração inicial.

O contrato de comunicação entre ARM e FPGA é baseado em duas filas circulares: uma para eventos e outra para respostas. Cada fila é de produtor único e consumidor único. A convenção adotada reserva uma posição para distinguir fila cheia de fila vazia, o que simplifica a lógica de controle e evita contador compartilhado entre os dois domínios.

2.6.1 Ordenação de publicação

A ordem de publicação é uma propriedade de corretude. No envio ARM–FPGA, o software deve escrever todo o evento antes de publicá-lo. Na recepção FPGA–ARM, o software deve ler toda a resposta antes de liberar o espaço. Caso essa ordem seja invertida, o sistema pode consumir dados incompletos ou sobrescrever respostas ainda não lidas.

Essa combinação é simples, mas exige que documentação, software e hardware permaneçam alinhados. Por isso, o contrato compartilhado é tratado como uma superfície crítica do sistema.

2.6.2 Avalon-MM como camada de integração

Avalon-MM é usado para conectar o motor HFT ao HPS no Platform Designer. O sinal de espera do protocolo permite que o periférico indique quando uma transação ainda não foi concluída. Se esse sinal permanecer ativo indefinidamente, o processador pode ficar bloqueado ao acessar o periférico. Por isso, a integração inclui tratamento de timeout como mecanismo de robustez durante depuração.

2.6.3 MMIO versus DMA

O uso de MMIO com polling é apropriado para simplicidade e depuração. No Cyclone V SoC, a documentação da Intel recomenda a *lightweight HPS-to-FPGA bridge* para acessos de controle e status, enquanto a ponte HPS-to-FPGA de maior largura é indicada para memória e transferências em rajada (Intel Corporation, 2022). Como o protótipo expõe um periférico de comando e resposta com filas pequenas, e não uma memória de dados de alta largura de banda, a escolha por MMIO é coerente com o papel da interface no sistema.

Além disso, a campanha experimental mostrou que MMIO já atende a meta de vazão do TCC. O benchmark mediu 102.381 mensagens por segundo no caminho ARM→FPGA→ARM. Considerando apenas o payload útil de um evento e uma resposta, cada mensagem movimenta 64 bytes, o que corresponde a aproximadamente 6,55 MB/s. Mesmo incluindo os acessos extras de ponteiros e registradores, a carga permanece muito abaixo da capacidade bruta inferida para uma interface de 32 bits a 50 MHz, da ordem de 200 MB/s. Portanto, no escopo deste trabalho, MMIO não se mostrou um gargalo de throughput.

O custo real dessa escolha aparece em outra dimensão: latência fim a fim e uso de CPU no ARM. Cada mensagem exige múltiplos acessos individuais a registradores e slots, o que aumenta o RTT observado no Linux embarcado. Ainda assim, como a meta de vazão foi cumprida com folga e o sistema utiliza apenas um canal de comunicação, a introdução de DMA deixaria de ser uma necessidade do protótipo atual e passaria a ser apenas uma alternativa arquitetural para versões futuras com quadros maiores, múltiplos canais ou integração com buffers de memória de maior volume.

Latência é o intervalo entre a chegada de um evento e a disponibilidade da resposta correspondente. Vazão é a quantidade de eventos processados por unidade de tempo. Em sistemas de baixa latência, o valor médio não é suficiente; variância, piores casos e caudas

de distribuição são igualmente importantes. Uma solução com latência média baixa, mas picos imprevisíveis, pode ser inadequada para HFT.

No protótipo, a latência total resulta da combinação entre recepção do feed, normalização no ARM, escrita MMIO, processamento em FPGA e leitura da resposta. Já a latência interna da FPGA corresponde apenas ao trecho lógico dedicado ao processamento do evento. Essa distinção é importante porque o sistema completo e o núcleo em hardware possuem comportamentos temporais diferentes, como será mostrado nos resultados.

3 Trabalhos Relacionados

Os trabalhos relacionados são agrupados em três eixos: efeitos e características de negociação de baixa latência, aceleração por FPGA em HFT e modelagem de Order Book. Essa separação ajuda a organizar a revisão entre contexto de mercado, técnicas de implementação e estruturas de dados relevantes.

3.1 Negociação algorítmica e baixa latência

Hendershott, Jones e Menkveld (2011) analisam a relação entre negociação algorítmica e liquidez, indicando que automação pode melhorar algumas métricas de mercado ao reduzir custos de seleção adversa e aumentar eficiência de cotação. Brogaard, Hendershott e Riordan (2014) discutem HFT e descoberta de preços, mostrando que participantes de alta frequência podem contribuir para incorporação de informação aos preços, embora o debate permaneça dependente de contexto e desenho de mercado. Hasbrouck e Saar (2013) tratam especificamente de negociação de baixa latência, destacando que a velocidade passa a ser um componente estratégico da interação entre participantes.

Esses estudos justificam a relevância de investigar infraestrutura computacional. Eles, porém, concentram-se em análise econômica e estatística, não em mecanismos de implementação de uma cadeia ARM-FPGA para processamento de dados de mercado.

3.2 Aceleração de HFT com FPGA

Leber, Geib e Litz (2011) apresentam aceleração de HFT com FPGAs, destacando a possibilidade de deslocar funções sensíveis a latência para hardware reconfigurável. O argumento central é próximo ao adotado neste TCC: operações de mercado com formato fixo e fluxo repetitivo podem se beneficiar de circuitos dedicados. Boutros *et al.* (2017) exploram HFT baseado em FPGA com HLS, demonstrando que ferramentas de síntese de alto nível podem ser usadas para construir blocos de negociação com menor esforço de desenvolvimento em comparação à escrita manual integral em RTL. Nesse trabalho, o sistema foi implementado em uma Xilinx Kintex UltraScale rodando a 156 MHz, com latência de ida e volta inferior a 870 ns e latência de 269 ns para o subsistema HFT; por simples inversão dessa latência, isso equivale a uma taxa teórica de aproximadamente 3.717.472 mensagens por segundo para o núcleo lógico. Lockwood *et al.* (2012) discutem uma biblioteca de baixa latência em FPGA para HFT, reforçando que a latência não depende apenas do algoritmo, mas também da forma de conexão entre rede, memória, barramento e lógica.

A diferença principal em relação ao presente trabalho é o escopo de custo, ferramenta e maturidade. Os trabalhos citados se aproximam de bibliotecas e plataformas de maior desem-

penho, enquanto este TCC utiliza uma DE10-Nano e privilegia um fluxo acadêmico em que a estratégia é escrita em MATLAB e gerada por HDL Coder. Em vez de implementar MAC Ethernet, UDP multicast ou *kernel bypass*, o projeto usa TCP sintético e concentra a aceleração no Order Book e na estratégia. Essa escolha reduz fidelidade operacional, mas torna o projeto reproduzível em laboratório de graduação.

Essa diferença também aparece no orçamento e na plataforma física. A DE10-Nano utilizada neste trabalho custa aproximadamente US\$190 em condição acadêmica (Terasic Technologies, 2026). Já o FPGA AMD/Xilinx XCKU115-2FLVA1517E, usado por Boutros *et al.* (2017), aparece listado por US\$10.290,57 na Mouser (Mouser Electronics, 2026). Em outras palavras, apenas o componente lógico de alto desempenho já custa mais de 54 vezes o valor da plataforma completa adotada neste TCC. Além disso, Boutros *et al.* (2017) operam com uma infraestrutura diretamente conectada à rede e com clock de 156 MHz, ao passo que o presente protótipo opera a 50 MHz e depende da cadeia ARM→MMIO→FPGA→MMIO→ARM para fechar o laço de decisão. Portanto, uma comparação puramente por throughput bruto seria metodologicamente injusta se não considerar simultaneamente custo, frequência e arquitetura de comunicação.

Do ponto de vista de arquitetura de computadores, a motivação para FPGA também pode ser interpretada pela diferença entre execução sequencial e execução espacial. Em uma CPU, uma sequência de atualizações do Order Book compartilha unidades funcionais, hierarquia de cache e controle de fluxo. Mesmo quando há paralelismo interno, como execução fora de ordem, predição de desvios e vetorização, o programador observa uma abstração sequencial. Na FPGA, comparadores, registradores e somadores podem ser alocados como estruturas físicas distintas, permitindo que partes do processamento ocorram simultaneamente. Essa vantagem, contudo, só aparece quando o problema possui regularidade suficiente para justificar hardware dedicado (PATTERSON; HENNESSY, 2017; HARRIS; HARRIS, 2013).

O problema estudado se encaixa nesse perfil porque o quadro de entrada tem largura fixa, a profundidade do Order Book é parametrizada e a estratégia usa limiares simples. A parte menos regular, isto é, decodificação de mensagens FAST, tratamento de TCP e manipulação de strings, permanece em software. Essa divisão evita transferir para a FPGA tarefas que exigiriam muito controle e pouco ganho.

3.3 Order Book e modelos de atualização

Gould *et al.* (2013) revisam conceitos fundamentais de Order Books, incluindo melhor compra, melhor venda, profundidade, eventos e impacto de chegada de ordens. Cont, Stoikov e Talreja (2010) propõem modelagem estocástica da dinâmica do livro, indicando que chegadas, cancelamentos e execuções podem ser tratados como processos que alteram estado discretamente. Embora o protótipo não implemente modelo estocástico completo nem prioridade temporal real, ele adota o princípio de estado discreto atualizado por eventos.

O Order Book adotado na plataforma é propositalmente limitado: os eventos são de nível de preço, a quantidade substitui o valor agregado no nível, a profundidade é fixa e o número de símbolos é parametrizado. Essa escolha permite implementação em VHDL com arrays estáticos e laços limitados. Em comparação com estruturas de software mais gerais, há perda de expressividade; em comparação com uma estrutura de hardware totalmente customizada para um mercado específico, há ganho de clareza e facilidade de teste.

3.4 Síntese de alto nível e fronteiras de projeto

HLS é uma técnica recorrente em projetos de aceleração porque permite expressar parte do comportamento em linguagem de nível mais alto, reduzindo o esforço de escrita RTL. O MATLAB HDL Coder segue essa linha ao converter funções compatíveis com síntese em VHDL ou Verilog (The MathWorks, Inc., 2026b). Entretanto, a literatura e a prática de projeto indicam que HLS não elimina a necessidade de decisões arquiteturais: tipos de dados, latência, paralelismo, interface, memória e protocolo continuam determinando a qualidade do circuito gerado.

Na plataforma proposta, essa fronteira aparece de forma clara. A estratégia é escrita em MATLAB, pois consiste em comparações e condições sobre sinais já normalizados. Já a ponte MMIO, as filas circulares e o Order Book foram mantidos em VHDL manual. Essa escolha é compatível com o papel de cada bloco: a estratégia é uma política que tende a mudar com experimentos; a ponte é infraestrutura crítica que exige controle explícito de registradores, ponteiros e ordem de publicação. O uso parcial de HLS, portanto, não representa uma limitação acidental, mas uma decisão de engenharia.

Em relação a Boutros *et al.* (2017), a diferença principal está menos na busca por uma implementação HFT completa e mais na construção de um fluxo acessível para graduação. A ferramenta escolhida é MATLAB HDL Coder, o que favorece a escrita de estratégias em uma linguagem já usada em modelagem numérica.

3.5 Comparação com o protótipo desenvolvido

O Quadro 1 sintetiza a relação entre os trabalhos revisados e o protótipo. O objetivo não é estabelecer superioridade, mas evidenciar o espaço ocupado por este TCC: uma plataforma acadêmica, aberta, de baixo custo e orientada à integração hardware/software e à geração de estratégias por HLS.

3.6 Posicionamento do trabalho

O protótipo desenvolvido posiciona-se como uma ponte entre trabalhos conceituais de microestrutura e sistemas profissionais de baixa latência. Sua contribuição acadêmica está em

Quadro 1 – Comparação entre eixos da literatura e o protótipo

Eixo	Ênfase na literatura	Tratamento neste TCC
Microestrutura e HFT	Efeitos de baixa latência sobre liquidez, descoberta de preço e competição entre agentes.	Uso do contexto de HFT como motivação computacional, sem pretensão de avaliar lucratividade ou impacto econômico.
Aceleração por FPGA	Redução de latência em funções críticas, frequentemente com rede especializada, plataformas profissionais e HLS em C/C++. Ex.: Kintex UltraScale a 156 MHz e latência sub-microsegundo em Boutros <i>et al.</i> (2017).	Uso de DE10-Nano de US\$190 em condição acadêmica, MMIO, filas circulares, livro simplificado e estratégia MATLAB gerada por HDL Coder.
Order Book	Modelos por ordem, por nível, estoquísticos ou estatísticos, com eventos de chegada, cancelamento e execução.	Livro por nível agregado, profundidade fixa, múltiplos símbolos parametrizados e proteção contra livro cruzado.
HLS	Aumento de produtividade na geração de RTL, condicionado a restrições de código sintetizável.	Aplicação central à criação de estratégias em MATLAB, mantendo transporte e estado de mercado em VHDL manual.
Validação	Medições de latência, frequência, vazão, área e comparação com alternativas.	Validação funcional por testes C++ e testbenches VHDL, complementada por benchmark quantitativo em placa para latência, vazão e speedup do núcleo lógico.

Fonte: Autoria própria (2026).

explicitar, em código aberto e com ferramenta acessível, como organizar a fronteira entre ARM e FPGA em uma aplicação de mercado. O trabalho evidencia contratos que costumam ser pouco visíveis em descrições abstratas: largura de quadro, ordem de palavras, ponteiros de fila, base dinâmica da região RX, tratamento de livro cruzado e validação por testbench.

Assim, a principal originalidade local está na integração entre feed sintético, receptor C++ com mFAST, wrapper MMIO, ponte VHDL de anéis, Order Book simplificado, estratégia gerada por MATLAB HDL Coder e componente Avalon-MM para DE10-Nano. A lacuna coberta é sobretudo pedagógica e experimental: poucos materiais de graduação mostram em detalhe como essa cadeia completa é organizada e validada.

4 Proposta

Este capítulo apresenta a proposta da plataforma: uma infraestrutura de baixo custo para experimentar estratégias de trading escritas em MATLAB e geradas para FPGA por meio do MATLAB HDL Coder. A arquitetura não busca implementar um sistema completo de HFT em produção; ela prepara o caminho de dados, o Order Book, a comunicação ARM–FPGA e a validação para que o bloco de estratégia possa ser substituído com menor esforço.

4.1 Visão geral da arquitetura

O fluxo principal inicia em um gerador de mercado sintético, que publica mensagens por TCP com símbolo, lado, preço, quantidade e número de sequência. Um receptor decodifica essas mensagens, imprime os dados e, quando o modo FPGA está habilitado, normaliza os campos para uma representação numérica fixa. Essa normalização inclui conversão de símbolo textual para identificador inteiro, preço decimal para ponto fixo e lado da operação para código numérico.

No FPGA, a ponte de comunicação recebe os eventos por filas circulares e os entrega ao motor de decisão. Esse motor atualiza o Order Book, calcula melhor compra, melhor venda, *spread* e desbalanceamento, e aciona a estratégia gerada a partir de MATLAB. A resposta retorna ao software com a ação decidida e os campos derivados do livro.

Essa arquitetura possui três fronteiras explícitas. A primeira fronteira é protocolo–software: bytes FAST são transformados em objetos C++ tipados. A segunda é software–hardware: objetos C++ são transformados em palavras de 32 bits e publicados por MMIO. A terceira é estado–decisão: eventos brutos são transformados em sinais derivados de livro, e esses sinais alimentam a estratégia. A separação em fronteiras facilita a validação porque cada etapa pode ser testada isoladamente.

A Figura 4 mostra os blocos principais do sistema organizados em três camadas. No fluxo de desenvolvimento (offline), a estratégia de trading é escrita em MATLAB, convertida pelo HDL Coder e integrada à FPGA em tempo de síntese (seta tracejada). Na camada de software, um gerador produz dados sintéticos de mercado, que são decodificados e normalizados antes de chegarem à interface MMIO. Na FPGA, os eventos percorrem um pipeline de três estágios: as filas de comunicação recebem os eventos do ARM e devolvem as respostas, o Order Book atualiza o estado por símbolo e calcula os sinais de topo, e a estratégia gerada por HLS produz a ação de trading.

Essa organização pode ser entendida em três grupos. O primeiro grupo é o software de apoio, responsável por produzir dados sintéticos, decodificar mensagens financeiras e transformar informações textuais em números adequados ao hardware. O segundo grupo é o hardware reconfigurável, responsável por transportar eventos, manter o Order Book e executar a decisão.

O terceiro grupo é a infraestrutura de desenvolvimento, que reúne build, simulação, geração HLS, integração Quartus e documentação.

Essa separação evita que o usuário interessado em testar uma estratégia precise alterar todo o sistema. O caminho de dados, a ponte ARM–FPGA, os testes e a integração com a placa formam uma base relativamente estável. A parte que deve variar com maior frequência é a estratégia em MATLAB, que recebe sinais já preparados pelo Order Book e retorna uma ação.

4.2 Decisões arquiteturais

A primeira decisão arquitetural é manter a decodificação FAST no ARM. O protocolo FAST depende de templates, campos opcionais e representação binária compacta. Implementá-lo diretamente em FPGA exigiria uma máquina de estados maior, tratamento de variabilidade de mensagem e manutenção cuidadosa de compatibilidade com o template. Ao manter essa etapa em C++, o projeto preserva flexibilidade e usa a FPGA onde o formato já está normalizado.

A segunda decisão é usar filas circulares em memória mapeada em vez de um conjunto de registradores para cada campo. Essa escolha desacopla parcialmente o tempo do ARM e do FPGA: o software pode publicar eventos enquanto houver espaço, e o hardware pode consumi-los no seu próprio ritmo. O custo é manter regras claras de publicação e consumo, mas esse custo é aceitável porque o contrato fica estável e testável.

A terceira decisão é separar Order Book e estratégia. O livro é estado persistente; a estratégia é política de decisão. Misturar ambos em um único bloco dificultaria testar atualização de livro sem acionar regra de negociação, ou testar a regra com snapshots artificiais. A separação permite substituir a estratégia gerada por MATLAB sem reescrever o Order Book em VHDL. Também permite, futuramente, comparar múltiplas estratégias sobre o mesmo estado de mercado.

A quarta decisão é usar tamanhos fixos para filas, símbolos e profundidade do livro. Em FPGA, memória e lógica precisam ser definidas antes da síntese; portanto, estruturas dinâmicas típicas de software não são adequadas. A limitação é que mudanças de capacidade exigem nova síntese, mas a vantagem é obter um circuito previsível e compatível com validação por testbench.

4.3 Fluxo de criação de estratégias com MATLAB HDL Coder

O objetivo principal da plataforma é reduzir o esforço necessário para implementar uma nova estratégia de trading em FPGA. Para isso, o ambiente separa a infraestrutura fixa da política de decisão. A infraestrutura fixa inclui recepção de mensagens, normalização, transporte MMIO, filas circulares, Order Book simplificado, wrapper Avalon-MM e testbenches. A política de decisão é concentrada em uma função MATLAB, representada por `strategy.m`.

O fluxo esperado para o usuário é:

1. escrever ou alterar a função `strategy.m`, recebendo sinais numéricos do Order Book, como melhor compra, melhor venda, quantidades, *spread* e desbalanceamento;
2. respeitar restrições de código sintetizável, como uso de tipos inteiros ou ponto fixo, ausência de alocação dinâmica, ausência de strings e controle de fluxo com limites conhecidos;
3. executar o fluxo de geração para obter o bloco VHDL por meio do MATLAB HDL Coder;
4. integrar o bloco gerado ao motor de decisão, mantendo a mesma interface de entrada e saída da plataforma;
5. executar os testbenches da estratégia e do motor completo para verificar se BUY, SELL e NOOP ocorrem nos cenários esperados;
6. somente após validação funcional, seguir para síntese Quartus e coleta de métricas.

Essa organização desloca o esforço do usuário para a definição algorítmica. Em vez de alterar filas, registradores, barramentos ou empacotamento de quadros, o usuário escreve uma regra em MATLAB dentro de uma assinatura conhecida. A vantagem é produtividade: estratégias simples podem ser descritas de forma mais próxima da notação algorítmica e depois convertidas para hardware. A limitação é que nem todo código MATLAB pode ser sintetizado. Estratégias com estruturas dinâmicas, bibliotecas não suportadas, aprendizado de máquina complexo ou uso intenso de ponto flutuante exigiriam adaptação para tipos fixos e arquitetura compatível com FPGA.

4.4 Modos de operação e delimitações

O protótipo possui dois modos práticos. No modo software-only, o receptor decodifica e imprime mensagens sem acessar FPGA. Esse modo é útil para validar o feed, o protocolo FAST e a normalização dos dados. Também serve como referência inicial para futuras comparações de latência.

No modo FPGA, o receptor acessa o periférico mapeado em memória, valida se a ponte esperada está presente e passa a enviar eventos ao hardware. Se a ponte não estiver acessível, a falha ocorre no início, evitando que o programa opere sobre uma região incorreta.

Essa dualidade é metodologicamente útil. O modo software-only permite testar o lado de mercado em qualquer máquina de desenvolvimento. O modo FPGA exige placa, bitstream e permissões de acesso MMIO, mas valida a contribuição central do trabalho. A existência dos dois modos reduz o tempo de depuração, pois um erro de FAST não precisa ser confundido com erro de barramento.

O sistema também possui mecanismos simples de observabilidade: identificação do periférico, versão de protocolo, geometria da fila e mensagens de erro no receptor. Esses mecanismos são suficientes para o estágio de protótipo, pois ajudam a diferenciar falhas de feed, falhas de decodificação e falhas de comunicação com a FPGA sem transformar o texto em uma descrição exaustiva de registradores.

O escopo positivo inclui geração sintética de dados, decodificação FAST, transporte ARM–FPGA, Order Book simplificado, decisão por estratégia gerada e preparação para execução na DE10-Nano. O escopo negativo inclui conexão real com bolsa, envio de ordens, controle de risco profissional, dados proprietários, multicast real, *kernel bypass*, conformidade regulatória e métricas finais de latência com instrumentação dedicada. Essas delimitações mantêm o trabalho adequado a uma monografia de graduação.

4.5 Contrato de eventos ARM–FPGA

O quadro ARM–FPGA possui oito palavras de 32 bits, totalizando 256 bits. A estrutura adotada é resumida na Tabela 1.

Tabela 1 – Quadro de evento enviado do ARM para o FPGA

Palavra	Campo	Tipo	Descrição
0	seq_no	uint32	Número de sequência da mensagem sintética.
1	symbol_id	uint32	Identificador inteiro do símbolo, mapeado no ARM.
2	price_1e4	uint32	Preço multiplicado por 10^4 .
3	qty	uint32	Quantidade agregada no nível.
4	event_type	uint32	1 para inserção/atualização, 2 para remoção, 3 para reinicialização.
5	side	uint32	1 para compra e 2 para venda.
6	level_hint	uint32	Reservado para indicação futura de nível ou ordem.
7	reserved	uint32	Reservado para extensão futura.

Fonte: Autoria própria (2026).

O quadro de resposta FPGA–ARM também possui oito palavras:

Esse contrato é o eixo de integração do trabalho. Qualquer mudança na largura do quadro, no significado das palavras ou na ordem de publicação deve ser refletida simultaneamente em C++, VHDL, testes e documentação.

4.6 Mapeamento de símbolos e geração de dados

O receptor traduz símbolos textuais para identificadores inteiros antes de enviar o evento à FPGA. Essa decisão reduz a complexidade do hardware, que passa a operar apenas com números. O preço também é convertido para uma escala inteira, evitando ponto flutuante no circuito.

Tabela 2 – Quadro de resposta enviado do FPGA para o ARM

Palavra	Campo	Tipo	Descrição
0	seq_no	uint32	Sequência ecoada para correlação.
1	action	uint32	0 para NOOP, 1 para BUY e 2 para SELL.
2	best_bid_px	uint32	Melhor preço de compra em escala 10^4 .
3	best_bid_qty	uint32	Quantidade no melhor nível de compra.
4	best_ask_px	uint32	Melhor preço de venda em escala 10^4 .
5	best_ask_qty	uint32	Quantidade no melhor nível de venda.
6	spread_1e4	uint32	Diferença entre melhor venda e melhor compra.
7	imbalance	int32	Diferença assinada entre quantidade de compra e venda.

Fonte: Autoria própria (2026).

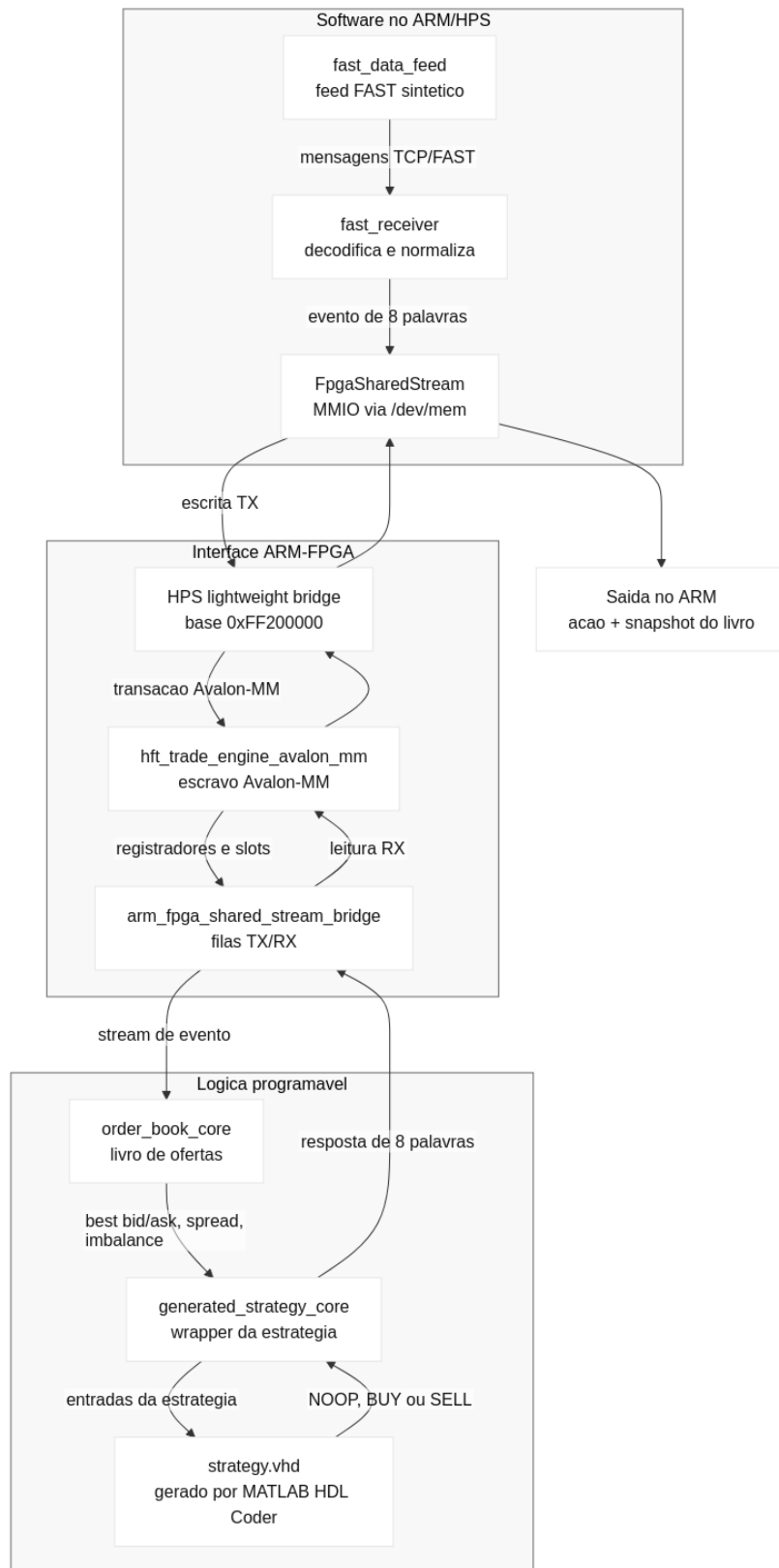
O gerador de dados cria preços próximos a um valor base por símbolo e alterna lados e quantidades de forma pseudoaleatória. O objetivo não é reproduzir fielmente uma bolsa, mas fornecer estímulos suficientes para validar livro, estratégia e comunicação.

4.7 Regra de estratégia

A estratégia atual é intencionalmente simples. Se um dos lados do livro está vazio, se o livro está cruzado ou se o *spread* supera 2,5000 unidades monetárias em escala 10^4 , a ação é NOOP. Se o desbalanceamento é maior ou igual a 500, a ação é BUY. Se o desbalanceamento é menor ou igual a -500, a ação é SELL. Caso contrário, a ação permanece NOOP.

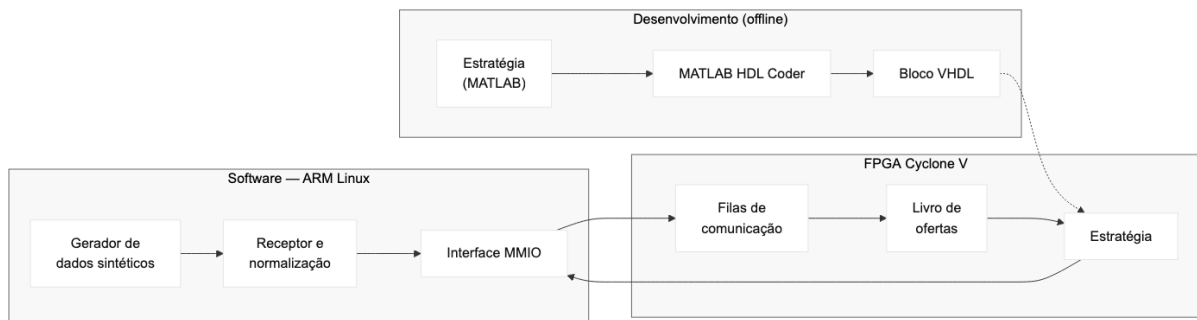
Essa regra não deve ser avaliada como estratégia financeira lucrativa. Sua finalidade é produzir uma decisão verificável e dependente do estado do Order Book, permitindo testar se o pipeline de dados preserva campos, ordem de palavras, sinais e semântica. A vantagem é a simplicidade para validação; a limitação é a baixa representatividade de estratégias reais, que exigiriam controle de risco, previsão, custos, posição, *slippage* e conformidade.

Figura 3 – Arquitetura geral da plataforma proposta



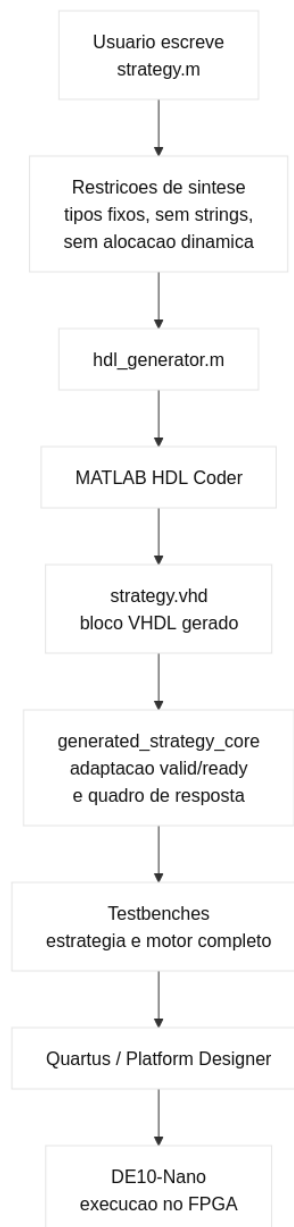
Fonte: Elaborado pelos autores a partir do diagrama Mermaid de arquitetura do projeto..

Figura 4 – Diagrama de blocos do sistema



Fonte: Elaborado pelos autores..

Figura 5 – Fluxo sugerido de criação de estratégia por MATLAB HDL Coder



Fonte: Elaborado pelos autores a partir de diagrama Mermaid..

5 Materiais e Métodos

Este capítulo descreve os materiais e os métodos utilizados para construir e validar a plataforma proposta. A ênfase metodológica está no uso de MATLAB HDL Coder para gerar a lógica de estratégia, enquanto a infraestrutura de comunicação, Order Book e testes permanece como base de execução.

5.1 Materiais

A plataforma alvo é a DE10-Nano, baseada em Intel Cyclone V SoC. A escolha dessa placa é adequada ao trabalho porque combina processador ARM, FPGA e interface HPS–FPGA em um ambiente de baixo custo. O ARM executa Linux embarcado, sockets e decodificação de mensagens; a FPGA executa os blocos que exigem paralelismo e comportamento temporal mais previsível.

O software de apoio foi desenvolvido em C++ e utiliza a biblioteca mFAST para codificação e decodificação de mensagens FAST. Essa camada é responsável por gerar dados sintéticos, receber mensagens, normalizar símbolos, converter preços para ponto fixo e interagir com a região de memória mapeada da FPGA. Essa escolha mantém no software as partes mais variáveis do protocolo de mercado e envia ao hardware apenas dados numéricos.

O hardware foi descrito em VHDL, com simulações também em VHDL. A infraestrutura manual inclui a ponte ARM–FPGA, o Order Book, o empacotamento de respostas e a adaptação ao barramento Avalon-MM. A estratégia de trading, por sua vez, é escrita em MATLAB e convertida para VHDL por meio do HDL Coder. Essa divisão foi adotada para preservar controle explícito das interfaces críticas e, ao mesmo tempo, facilitar a criação de novas regras de decisão.

O fluxo de construção e validação utiliza Make, Docker, CMake, GHDL, MATLAB HDL Coder e Quartus Prime Lite com Platform Designer. Essas ferramentas são empregadas para reduzir diferenças de ambiente, executar testes C++ e VHDL, gerar o bloco HDL da estratégia, compilar o projeto FPGA e preparar binários para a DE10-Nano. Como o trabalho depende de ferramentas acadêmicas e dados sintéticos, não há necessidade de acesso a uma bolsa real, contrato de dados proprietário ou infraestrutura profissional de negociação.

5.2 Métodos

5.2.1 Organização da plataforma

O método de desenvolvimento foi orientado pela separação entre infraestrutura e estratégia. A infraestrutura compreende recepção de dados, normalização, transporte por MMIO,

Order Book, integração Avalon-MM e testes. A estratégia é mantida em uma função MATLAB sintetizável, com entradas numéricas já derivadas do Order Book. Assim, a troca de estratégia deve exigir alteração prioritariamente no código MATLAB, não na ponte de comunicação nem no Order Book.

Essa organização também define a responsabilidade de cada domínio. O software lida com mensagens textuais ou binárias de formato flexível. O hardware lida com palavras de largura fixa, estado do livro e decisão. O MATLAB HDL Coder atua no ponto de maior interesse do usuário: a regra de trading. A consequência é um fluxo de trabalho mais simples para experimentar variações de estratégia, desde que sejam respeitadas as restrições de código sintetizável.

5.2.2 Integração no Quartus e Platform Designer

Além da descrição RTL e da geração automática da estratégia, o trabalho exige uma etapa explícita de integração no Quartus Prime Lite, em especial no Platform Designer da Intel (Intel Corporation, 2024b). É nessa ferramenta que o HPS, a ponte HPS–FPGA e o periférico customizado `hft\_trade\_engine\_avalon\_mm` são ligados em um sistema SoC coerente para a DE10-Nano.

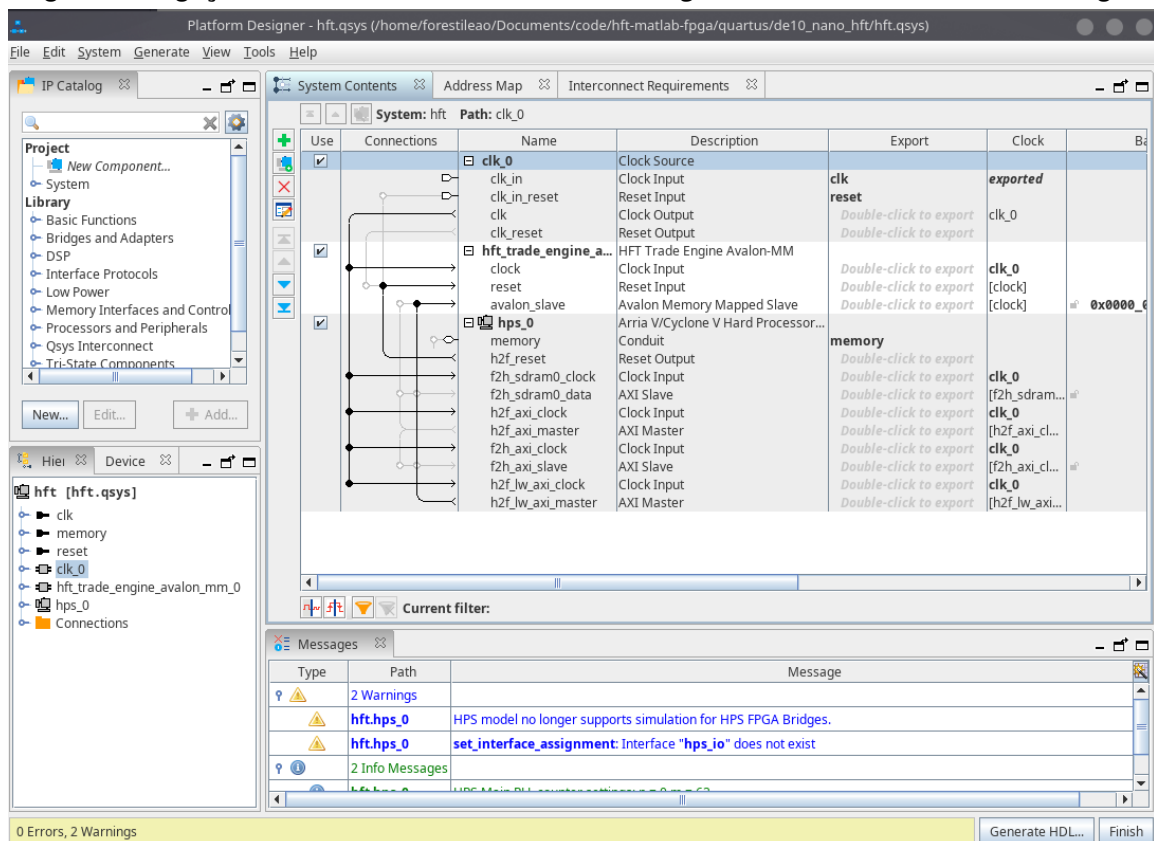
A Figura 6 mostra a integração utilizada no projeto. Nela, o HPS atua como bloco iniciador da comunicação e o motor HFT aparece como periférico Avalon-MM do tipo *slave*. Essa ligação é importante porque materializa, em nível de ferramenta, o contrato discutido ao longo do texto: o software ARM publica eventos pela ponte, o periférico no FPGA consome esses eventos, atualiza o Order Book e devolve respostas pela fila RX. Em outras palavras, a figura serve como evidência visual de como a arquitetura proposta foi efetivamente montada no ambiente Quartus.

Essa etapa de integração complementa os artefatos em VHDL e os scripts de build. Embora a lógica de decisão, a ponte ARM–FPGA e o Order Book estejam descritos nos arquivos-fonte, é o Platform Designer que define a conexão final com o HPS, o mapeamento Avalon-MM e a forma como o sistema sintetizado passa a existir como periférico acessível pelo software embarcado.

5.2.3 Contrato de comunicação

O contrato ARM–FPGA foi definido como uma interface de produtor e consumidor baseada em duas filas circulares: uma para eventos enviados pelo ARM e outra para respostas produzidas pela FPGA. Cada evento transporta sequência, símbolo normalizado, lado, preço em ponto fixo e quantidade. Cada resposta devolve sequência, ação, melhor compra, melhor venda, *spread* e desbalanceamento.

Figura 6 – Ligação entre HPS e o bloco HFT Trade Engine Avalon-MM no Platform Designer



Fonte: Captura de tela elaborada pelos autores no Quartus Prime Lite Platform Designer..

O uso de filas evita que cada evento dependa de uma chamada síncrona por registradores individuais. Com isso, o ARM pode publicar eventos enquanto houver espaço e a FPGA pode consumi-los conforme sua disponibilidade. Para manter corretude, o produtor escreve o conteúdo completo antes de publicar o ponteiro, e o consumidor lê o conteúdo completo antes de liberar o espaço. Os detalhes de offsets e palavras individuais são apresentados nas tabelas e apêndices, pois pertencem mais ao contrato técnico do que à discussão metodológica principal.

5.2.4 Fluxo de dados de mercado

O fluxo de dados usa mensagens sintéticas codificadas em FAST e transmitidas por TCP. A escolha por um feed sintético permite repetir testes sem depender de dados reais ou infraestrutura de mercado. O receptor decodifica as mensagens, converte campos para representação numérica e, quando a FPGA está habilitada, envia o evento normalizado ao hardware.

Essa abordagem não pretende medir a latência de uma rede de bolsa real. O objetivo é exercitar o caminho de dados até a estratégia: mensagem de mercado, normalização, transporte ARM–FPGA, atualização do livro e retorno da decisão. Assim, a metodologia separa validação funcional do pipeline de qualquer afirmação quantitativa sobre redes financeiras reais.

5.2.5 Order Book e estratégia

O Order Book foi implementado em hardware com profundidade limitada. Ele mantém níveis de compra e venda por símbolo, atualiza preços e quantidades, rejeita estados cruzados e calcula os sinais necessários à estratégia. A profundidade limitada é coerente com o objetivo do protótipo: validar a integração e fornecer sinais de topo de livro, não reproduzir integralmente a complexidade de um livro de bolsa.

A estratégia foi escrita em MATLAB com operações compatíveis com HDL Coder. Ela recebe os sinais do Order Book e retorna uma ação discreta: NOOP, BUY ou SELL. O papel do HDL Coder é transformar essa regra em lógica sintetizável, permitindo que o usuário trabalhe em um nível mais próximo da formulação algorítmica. Estratégias futuras podem alterar limiares ou combinar os sinais de outra forma, desde que mantenham tipos e operações adequados à síntese.

5.2.6 Validação funcional

A validação foi dividida por camadas. O software é validado por testes de comunicação e empacotamento. A ponte ARM–FPGA é validada por simulações de fila, rajadas e contrapressão. O Order Book é validado por casos de inserção, remoção, ordenação e rejeição de estados inválidos. A estratégia é validada por cenários que devem produzir NOOP, BUY e SELL. Por fim, o motor completo é validado pela passagem de eventos por todas as etapas.

O Quadro 2 resume os principais riscos cobertos por essa validação. A matriz evita que o trabalho dependa apenas de uma demonstração manual em placa, pois cada parte essencial da arquitetura possui uma evidência funcional própria.

Quadro 2 – Matriz de validação funcional do protótipo

Artefato	Risco técnico mitigado	Evidência de validação
Interface ARM–FPGA	Divergência de contrato, fila cheia mal detectada ou avanço de ponteiro fora de ordem.	Teste C++ e testbenches VHDL da ponte.
Order Book	Livro fora de ordem, remoção incorreta ou aceitação de livro cruzado.	Testbench específico com casos de inserção, atualização e remoção.
Estratégia gerada	Ação incompatível com os sinais de mercado ou resposta perdida.	Testbench da estratégia e verificação de NOOP em estados inválidos.
Motor integrado	Falha de integração entre comunicação, livro e decisão.	Testbench de motor completo com escrita de eventos e leitura de respostas.
Integração Avalon-MM	Barramento travado ou transação sem retorno durante acesso do HPS.	Testbench do wrapper Avalon-MM.
Feed e receptor	Falha de codificação, decodificação ou normalização dos eventos.	Teste de fumaça software-only e execução funcional em placa.

Fonte: Autoria própria (2026).

5.2.7 Método proposto para métricas experimentais

A implementação atual fornece principalmente evidências funcionais. Para completar a avaliação quantitativa, o método prevê instrumentar o software e a FPGA. No software, devem ser registrados os instantes de recebimento do evento, publicação do quadro e leitura da resposta. No hardware, recomenda-se usar um contador de ciclos para medir o intervalo entre consumo do evento e publicação da resposta.

A vazão deve ser medida aumentando progressivamente a taxa do feed até ocorrer saturação, fila cheia ou perda de resposta.

Quadro 3 – Métricas experimentais planejadas e resultados obtidos

Métrica	Como medir	Meta de referência	Resultado
Latência fim a fim (RTT)	Modo <code>fpga-sync</code> : mede RTT ARM→FPGA→ARM por mensagem com percentis.	Comparar com versão software-only.	10.228 ns médio; 10.310 ns p99 (Tabela 5)
Latência interna FPGA	Contador de ciclos do núcleo lógico.	Jitter interno inferior a 1 μ s.	60 ns, jitter 0 ns (seção 6.7)
Vazão	Eventos por segundo até ocorrência de saturação.	Pelo menos 100.000 mensagens/s.	102.381 msg/s
Speedup (núcleo lógico)	Comparação do núcleo de decisão FPGA vs. C++ equivalente (sem comun.).	Aceleração mínima de 5 vezes.	5,253× (lógica isolada)

Fonte: Autoria própria (2026).

As metas apresentadas funcionaram como critérios de avaliação para a campanha experimental e, na versão atual do trabalho, já podem ser confrontadas com resultados efetivamente obtidos. Essa distinção continua importante para não confundir integração funcional com desempenho: o benchmark já comprova aceleração do núcleo lógico e vazão da ponte MMIO.

6 Resultados

Este capítulo apresenta os resultados funcionais e quantitativos obtidos com a plataforma. A campanha de latência, vazão e aceleração já foi executada na DE10-Nano. Assim, os resultados aqui descritos devem ser interpretados tanto como evidência de integração e funcionamento do pipeline quanto como caracterização de desempenho do protótipo.

6.1 Plano da campanha experimental

A campanha experimental foi organizada para responder três perguntas diferentes sobre a plataforma. A primeira é funcional: *o sistema completo realmente opera em hardware real, da chegada do evento no ARM até a resposta produzida pela FPGA?* A segunda é estrutural: *cada camada da solução preserva o contrato esperado, ou a demonstração em placa funciona apenas em um cenário pontual?* A terceira é quantitativa: *o protótipo atende às metas de latência determinística, vazão mínima e aceleração em relação ao software?*

Para responder à primeira pergunta, foi realizado um ensaio de execução funcional na DE10-Nano. Esse experimento existe para comprovar que o caminho feed-ARM-FPGA-ARM foi de fato montado e executado na placa, e não apenas simulado. Ele é o experimento adequado para justificar afirmações de integração entre software, ponte MMIO, Order Book e estratégia em hardware.

Para responder à segunda pergunta, a validação foi organizada por testes em camadas. O motivo é simples: uma única execução em placa pode mostrar que o sistema funciona em um caso específico, mas não garante que filas, ordenação do Order Book, geração de ações e interface com o barramento estejam corretas em situações de contorno. Os testes isolados servem, portanto, para sustentar tecnicamente a robustez da infraestrutura construída.

Para responder à terceira pergunta, foi executado um benchmark quantitativo alinhado às métricas definidas na metodologia (Quadro 3). Esse benchmark foi dividido em duas visões complementares. A primeira mede apenas o núcleo lógico da FPGA, porque é essa medição que permite avaliar determinismo interno e aceleração do hardware em relação a uma implementação equivalente em software. A segunda mede o caminho completo ARM→FPGA→ARM, porque é essa medição que revela o custo prático da comunicação e explica a diferença entre o desempenho do núcleo lógico e o comportamento do sistema completo.

Há ainda uma quarta frente de análise neste capítulo: a eficiência energética. Diferentemente de latência, jitter, vazão e aceleração, essa parte não foi obtida por benchmark executado com instrumentação elétrica na placa. O que se apresenta é um experimento de cálculo, baseado em potência típica dos dispositivos e no tempo de processamento observado, com a finalidade de estimar a ordem de grandeza da energia por decisão. Essa distinção é importante para não atribuir o mesmo nível de evidência experimental a métricas medidas diretamente e a métricas inferidas por estimativa.

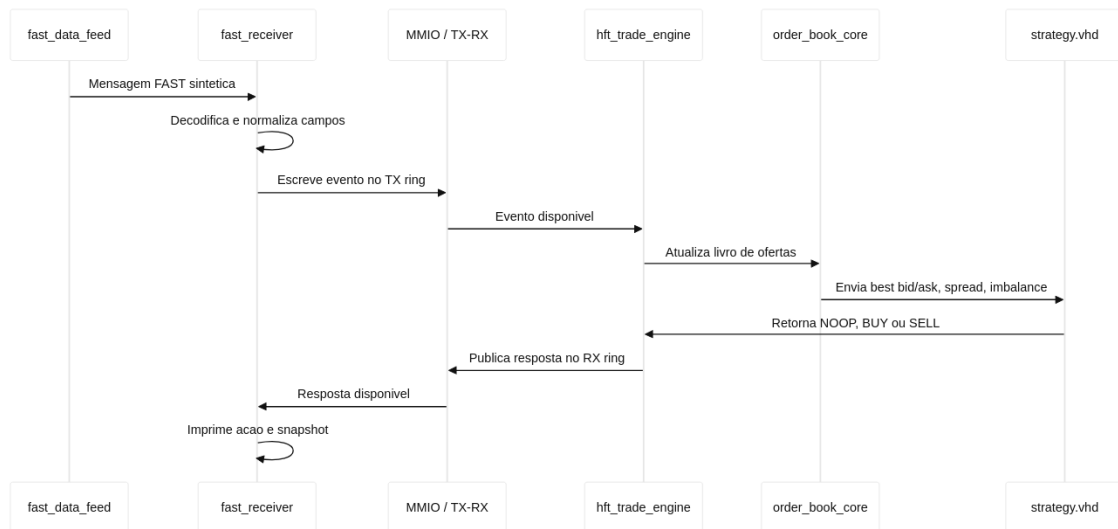
Essa divisão evita uma interpretação incorreta dos resultados. Quando o texto afirma que a FPGA apresenta jitter nulo, a afirmação refere-se ao núcleo lógico interno e foi obtida por benchmark executado na placa. Quando o texto discute RTT médio, p99 e cauda longa, a análise já inclui o custo da ponte MMIO e do ARM Linux e também foi obtida por benchmark. Já a discussão de energia deve ser lida como estimativa analítica, e não como medição elétrica direta. Em outras palavras, cada experimento foi desenhado para responder uma pergunta específica, e as conclusões do capítulo devem ser lidas dentro desse escopo.

6.2 Sistema obtido

O sistema desenvolvido recebe dados sintéticos de mercado, normaliza os eventos no ARM, envia os dados para a FPGA, atualiza um Order Book simplificado e executa uma estratégia de trading gerada a partir de MATLAB. A resposta retorna ao ARM com a ação escolhida e com os principais sinais do Order Book: melhor compra, melhor venda, quantidades, *spread* e desbalanceamento.

A Figura 7 resume o fluxo obtido. A parte de software trata o protocolo de entrada e a conversão dos dados. A parte em FPGA executa o processamento de estado e decisão. A fronteira entre os dois domínios é uma interface de memória mapeada, escolhida por ser simples de validar e adequada à integração com a DE10-Nano.

Figura 7 – Fluxo de dados do protótipo



Fonte: Elaborado pelos autores a partir de diagrama Mermaid..

Os requisitos funcionais atendidos foram: geração de mensagens sintéticas, decodificação FAST, envio de eventos normalizados ao hardware, manutenção de Order Book por símbolo, cálculo de sinais de topo de livro, geração das ações NOOP, BUY e SELL, retorno de resposta ao software e validação por testes em camadas. O sistema ainda não cobre negociação real, ro-

teamento de ordens, controle de risco, dados proprietários, multicast de mercado, *timestamping* de precisão ou comparação estatística com dados reais de bolsa.

6.3 Modelagem funcional

O Order Book foi modelado com profundidade limitada e representação em ponto fixo. Preços decimais são convertidos para inteiros escalados por 10^4 , o que simplifica comparações e subtrações em hardware. Essa representação evita ponto flutuante na FPGA e é suficiente para os dados sintéticos utilizados. Em uma aplicação real, a escala deveria ser definida de acordo com o *tick size* e a faixa de preço de cada instrumento.

A atualização do livro segue um pipeline lógico: interpretar o evento, localizar o símbolo, atualizar ou remover nível, preservar a ordenação, calcular o topo do livro e entregar os sinais à estratégia. A Figura 8 ilustra esse fluxo. A profundidade limitada reduz o custo lógico e facilita a validação, mas restringe estratégias que dependam de muitos níveis de profundidade.

A estratégia atual usa uma regra simples baseada em *spread* e desbalanceamento. Quando o livro está inválido, vazio ou com *spread* acima do limite, a ação é NOOP. Quando o desbalanceamento comprador ultrapassa o limiar, a ação é BUY. Quando o desbalanceamento vendedor ultrapassa o limiar negativo, a ação é SELL. Essa regra não é proposta como estratégia lucrativa; sua função é fornecer um comportamento verificável para demonstrar o fluxo MATLAB–HDL–FPGA.

6.4 Execução funcional na DE10-Nano

Este experimento foi realizado para responder à pergunta mais básica do capítulo: o protótipo funciona de ponta a ponta em hardware real? Foi então executado um ensaio funcional na DE10-Nano acessada por SSH. O receptor foi iniciado com o caminho FPGA habilitado e reconheceu corretamente a ponte MMIO. A inicialização indicou assinatura `0x48465431`, versão 1, filas com profundidade 64, slots de oito palavras e base RX calculada como `0x900`. Em seguida, o receptor conectou-se ao feed local e passou a imprimir, para cada evento recebido, a resposta produzida pelo hardware.

O trecho do Listagem 1 mostra três casos representativos. Para TSLA e NVDA, o desbalanceamento positivo resultou em ação BUY. Para AAPL, o desbalanceamento observado não atingiu o limiar de decisão e a ação retornada foi NOOP. Esse comportamento é coerente com a regra implementada e confirma que o ARM recebeu da FPGA não apenas uma ação, mas também o estado derivado do Order Book.

Esse ensaio responde positivamente à questão de integração funcional: o caminho feed–ARM–FPGA–ARM foi executado com sucesso na placa e produziu respostas coerentes com a regra implementada. Ao mesmo tempo, ele delimita o que ainda não pode ser concluído a partir

Figura 8 – Pipeline lógico de atualização do livro



Fonte: Elaborado pelos autores a partir de diagrama Mermaid..

desse único experimento: não há, aqui, base suficiente para afirmar determinismo temporal, vazão máxima ou aceleração. Essas afirmações dependem do benchmark quantitativo apresentado adiante.

Listagem 1 – Trecho da saída do receptor com respostas FPGA–ARM

```

1 FPGA MMIO bridge enabled: base=0xff200000 span=0x2000
2 magic=0x48465431 version=1 tx_depth=64 rx_depth=64
3 slot_words=8 rx_base=0x900 mode=new
4 Connected to 127.0.0.1:9001
5
6 seq=26 sym=TSLA side=sell price=175.17 qty=1547
7 [FPGA->ARM] seq=26 action=BUY
8   best_bid_px_1e4=1749900 best_bid_qty=2836
9   best_ask_px_1e4=1750100 best_ask_qty=681
10  spread_1e4=200 imbalance=2155
11
12 seq=27 sym=NVDA side=sell price=875.13 qty=3861
13 [FPGA->ARM] seq=27 action=BUY
14   best_bid_px_1e4=8749800 best_bid_qty=3282
15   best_ask_px_1e4=8750100 best_ask_qty=2259
16   spread_1e4=300 imbalance=1023
17
18 seq=36 sym=AAPL side=sell price=185.75 qty=1795
19 [FPGA->ARM] seq=36 action=NOOP
20   best_bid_px_1e4=1849900 best_bid_qty=939
21   best_ask_px_1e4=1850300 best_ask_qty=818
22   spread_1e4=400 imbalance=121

```

Fonte: Execução funcional na DE10-Nano realizada pelos autores..

Tabela 3 – Resumo da execução funcional na DE10-Nano

Item observado	Evidência
Ponte reconhecida	Assinatura, versão e geometria foram lidas antes do envio de eventos.
Feed conectado	O receptor conectou-se ao servidor local de dados sintéticos.
Eventos processados	Sequências recebidas pelo software geraram respostas correspondentes do hardware.
Ações observadas	BUY para desbalanceamentos positivos elevados e NOOP para desbalanceamento baixo.
Campos retornados	Melhor compra, melhor venda, quantidades, <i>spread</i> e desbalanceamento foram impressos pelo ARM.

Fonte: Autoria própria (2026).

6.5 Validação por testes

A validação automatizada foi organizada por risco técnico porque a execução em placa, sozinha, não basta para sustentar a corretude da infraestrutura. Os testes de software verificam o contrato de filas sem exigir hardware real. Os testbenches VHDL verificam a ponte, o Order Book, a estratégia, o motor integrado e a interface Avalon-MM. Essa divisão existe para transformar a demonstração funcional em evidência de engenharia: em vez de afirmar apenas que

“funcionou na placa”, o trabalho mostra também que cada camada preserva seus invariantes de forma isolada.

Quadro 4 – Critérios de aceitação funcional

Critério	Evidência esperada
Identificação da ponte	Software reconhece assinatura, versão e geometria antes de enviar eventos.
Publicação correta	O produtor escreve o conteúdo completo antes de publicar o evento.
Resposta correta	O consumidor lê a resposta completa antes de liberar o espaço.
Livro ordenado	Compras permanecem em ordem decrescente e vendas em ordem crescente.
Estado inválido protegido	Atualizações que cruzariam o livro são rejeitadas ou não geram ordem.
Estratégia verificável	NOOP, BUY e SELL aparecem em cenários controlados.
Integração em placa	Eventos reais do feed sintético produzem respostas FPGA–ARM.

Fonte: Autoria própria (2026).

Os critérios do Quadro 4 foram atendidos no nível funcional. Esse resultado é importante porque amarra a segunda pergunta da campanha experimental: não basta a plataforma produzir uma resposta; é necessário mostrar que ela o faz preservando contrato, ordenação e consistência entre as camadas. Com a comunicação, o Order Book e os testes funcionando, a regra MATLAB pode ser modificada e regenerada sem redesenhar toda a arquitetura.

6.6 Discussão

O resultado obtido reforça a viabilidade de uma plataforma acadêmica para estudar HLS em estratégias de trading. O uso de FPGA não é apresentado como solução completa para HFT profissional, mas como ambiente de experimentação em que decisões simples podem ser descritas em MATLAB, sintetizadas para hardware e avaliadas dentro de um fluxo ARM–FPGA funcional.

A principal vantagem da abordagem é a separação entre infraestrutura e política de decisão. O usuário pode concentrar-se na regra de trading, enquanto a plataforma fornece dados normalizados, sinais de livro e caminho de retorno. A principal limitação é que o sistema ainda trabalha com dados sintéticos, Order Book simplificado e métricas quantitativas pendentes. Portanto, os resultados atuais demonstram funcionamento e integração, não superioridade estatística ou operacional sobre sistemas de software em produção.

Também permanecem riscos técnicos típicos de projetos FPGA. Um circuito correto em simulação pode falhar em frequência após síntese se o caminho crítico for longo. A interface MMIO pode elevar o custo de CPU no ARM ou a latência fim a fim caso o quadro cresça ou o número de canais aumente. A estratégia gerada por HLS pode exigir ajustes caso use operações não sintetizáveis ou tipos inadequados. O Quadro 5 resume esses pontos.

Quadro 5 – Riscos técnicos remanescentes e mitigação

Risco	Mitigação recomendada
Caminho crítico no livro MMIO elevando custo no ARM	Coletar relatório de temporização e inserir pipeline se necessário. Reavaliar a interface apenas se houver aumento de canais, de tamanho de quadro ou de volume de dados.
Feed pouco realista	Adicionar replay de dados históricos e rajadas controladas.
Regeneração HLS incompatível	Manter wrapper estável e executar testbench após cada geração.
Comparação incompleta	Implementar versão software-only equivalente e coletar distribuições de latência.

Fonte: Autoria própria (2026).

6.7 Benchmark de desempenho

Este experimento foi planejado para responder à terceira pergunta da campanha experimental: a plataforma atende, de forma mensurável, às metas de latência determinística, vazão mínima e aceleração em relação ao software? Para isso, foi executado um benchmark automatizado que mede a latência interna da FPGA, a vazão do sistema e a aceleração em relação a uma implementação equivalente em software. O benchmark é implementado no binário `fpga\_benchmark` e opera em modo *full*: executa o núcleo de software em C++ e, em seguida, envia os mesmos eventos pela ponte MMIO, coletando as respostas da FPGA.

Essa separação entre núcleo lógico e caminho completo é intencional. Medir apenas o caminho ARM→FPGA→ARM esconderia o ganho do hardware atrás do custo de comunicação. Medir apenas o núcleo da FPGA, por outro lado, esconderia o gargalo prático observado na integração com o ARM. O benchmark, portanto, foi estruturado para capturar as duas visões e permitir uma interpretação mais honesta dos resultados.

6.7.1 Metodologia de medição

O benchmark gera 1.000.000 eventos sintéticos cobrindo cinco símbolos, com preços, quantidades e lados variados. Antes da campanha medida, são processadas 10.000 mensagens de aquecimento para estabilizar o estado do Order Book e eliminar efeitos de inicialização. As mensagens de aquecimento são seguidas por um reinício explícito dos contadores de desempenho, garantindo que apenas a campanha medida contribua para os resultados.

A latência interna da FPGA é medida por um contador de ciclos residente no próprio circuito. Essa medição isola o tempo de processamento do núcleo lógico e elimina o custo de comunicação ARM–FPGA do resultado. A frequência de operação configurada é de 50 MHz, portanto cada ciclo corresponde a 20 ns.

A referência em software é produzida pelo mesmo processo: o binário executa o mesmo algoritmo de Order Book e estratégia em C++ puro, sem acesso à FPGA, e mede o tempo médio por evento. A referência usa os mesmos limiares, a mesma lógica de atualização por nível e o

mesmo cálculo de *spread* e desbalanceamento, tornando a comparação direta entre hardware e software.

A corretude dos resultados é verificada por *checksum* cruzado: software e FPGA processam os mesmos eventos e os *checksums* das respostas são comparados. A igualdade dos *checksums* confirma que os dois caminhos produzem saídas bit a bit idênticas.

6.7.2 Resultados obtidos

A Tabela 4 sintetiza as métricas centrais da campanha executada na DE10-Nano com 1.000.000 de mensagens (10.000 de aquecimento). A tabela já separa, de propósito, o que pertence ao núcleo lógico da FPGA e o que pertence ao sistema completo com comunicação MMIO.

Tabela 4 – Métricas de desempenho do benchmark na DE10-Nano

Escopo	Métrica	Descrição	Valor
Núcleo lógico	Latência	Ciclos FPGA on-chip (50 MHz), sem comunicação	60 ns (3 ciclos)
	Jitter	Max – min, contador on-chip, 1 M mensagens	0 ns
	Aceleração	Lógica FPGA vs. núcleo C++ equivalente	5,253×
Fim a fim (MMIO)	Latência total	RTT ARM→FPGA→ARM, modo <code>fpga-sync</code>	10.228 ns médio; 10.310 ns p99
	Jitter total	Variação do RTT (escalador ARM + MMIO)	1.136.220 ns (max – min)
Sistema	Vazão	Mensagens/s com filas em paralelo, modo <code>fpga-mmio</code>	102.381 msg/s

Fonte: Benchmark executado pelos autores na DE10-Nano com `fpga\_benchmark -mode full -messages 1000000 -warmup 10000..`

Os critérios de aceitação definidos na metodologia foram atendidos no escopo da lógica interna: jitter inferior a 1 μ s no núcleo FPGA (resultado: 0 ns), vazão superior a 100.000 mensagens/s (resultado: 102.381 msg/s) e aceleração mínima de 5 vezes (resultado: 5,253×). Essa é a base técnica para afirmar que o núcleo do hardware cumpre a meta proposta pelo trabalho. No sistema completo, entretanto, o jitter fim a fim não é nulo e passa a ser dominado pela comunicação entre ARM e FPGA. Portanto, o valor de 0 ns não deve ser lido como propriedade do sistema inteiro, mas exclusivamente como propriedade do pipeline lógico interno. A corretude foi verificada por *checksum* cruzado entre os dois caminhos, com igualdade bit a bit confirmada. A latência fim a fim com a ponte MMIO é analisada separadamente na subseção 6.7.4.

6.7.3 Determinismo e ausência de jitter

O resultado mais significativo do benchmark, para fins da métrica de latência determinística, é a ausência total de jitter na latência interna da FPGA: mínimo, máximo e média são iguais a 3 ciclos (60 ns) para todas as 1.000.000 mensagens medidas. Esse resultado decorre de uma propriedade fundamental de circuitos digitais síncronos, não de uma coincidência estatística.

Em uma CPU executando software, a latência de uma operação varia por múltiplos fatores independentes da lógica da aplicação: falhas de cache nos níveis L1, L2 e L3, predição incorreta de desvios, paginação de memória virtual, preempção pelo escalonador do sistema operacional, variações de frequência por gerenciamento de energia e contenção de barramento entre *cores*. Mesmo com técnicas avançadas como *kernel bypass*, *busy polling* e afinidade de CPU, a variabilidade de cauda permanece presente porque o processador é um recurso de propósito geral compartilhado entre processos e sujeito a eventos assíncronos.

Na FPGA, o pipeline do motor de decisão é um circuito síncrono dedicado. A cada borda de subida do clock de 50 MHz, o circuito executa as mesmas transições de estado: interpreta o evento recebido, localiza o símbolo no Order Book, atualiza preços e quantidades, calcula os sinais de topo do livro e avalia os limiares da estratégia. A profundidade do Order Book (8 níveis por símbolo) e o caminho combinacional da estratégia são fixos em tempo de síntese. Não há alocação dinâmica de memória, coleta de lixo, dependência de cache, escalonamento de *threads* ou variação de frequência durante a operação. Consequentemente, o número de ciclos entre a aceitação de um evento e a produção da resposta é determinístico para toda carga de trabalho compatível com a configuração sintetizada.

Neste protótipo, a latência de 3 ciclos reflete o caminho mínimo do *pipeline*: atualização do Order Book, cálculo dos sinais de topo e avaliação da estratégia. A constância desse valor para 1.000.000 de eventos confirma que o caminho crítico é menor que o período de clock e que não há variação estrutural no processamento.

Essa propriedade distingue a abordagem FPGA de alternativas puramente em software: não se trata apenas de uma latência menor em média, mas de uma latência garantidamente constante independente da carga do sistema operacional, da pressão de memória ou da contenção entre processos (PATTERSON; HENNESSY, 2017; HARRIS; HARRIS, 2013). Em aplicações de negociação de baixa latência, essa garantia determinística é operacionalmente mais valiosa do que uma média baixa com cauda longa (HASBROUCK; SAAR, 2013).

É importante delimitar o escopo desta afirmação. O jitter zero vale **exclusivamente para o pipeline lógico interno**. Tão logo a cadeia inclua um componente não síncrono — rede IP, ponte MMIO, barramento PCIe ou sistema operacional — o jitter passa a existir. No modo `fpga-sync` deste benchmark, o RTT ARM→FPGA→ARM já apresenta jitter não nulo devido às variações de escalonamento do ARM Linux e à latência variável dos acessos MMIO. Portanto, o jitter zero é uma propriedade do núcleo FPGA isolado, não do sistema como um todo.

6.7.4 Gargalo de comunicação e latência fim a fim

A aceleração de 5,253× mede apenas o núcleo lógico. Para compreender o desempenho do sistema completo, e para explicar por que a latência fim a fim não acompanha a latência interna da FPGA, é necessário incluir o custo da ponte de comunicação ARM–FPGA.

Na arquitetura implementada, cada mensagem exige que o ARM escreva 8 palavras de 32 bits na fila TX e, após a resposta ser produzida pelo hardware, leia 8 palavras da fila RX, totalizando cerca de 20 acessos MMIO. A ponte HPS-to-FPGA *lightweight* da DE10-Nano opera por mapeamento de memória (`/dev/mem + mmap`). A documentação oficial da Intel descreve essa ponte como uma interface fixa de 32 bits, indicada para controle e status, enquanto a ponte HPS-to-FPGA de maior largura é reservada para memórias e transferências em rajada (Intel Corporation, 2022). No nosso caso, isso significa que a escolha por MMIO foi adequada ao papel do periférico, mas cobra um preço em número de acessos individuais realizados pelo ARM.

O modo `-mode fpga-sync` do benchmark instrumenta esse caminho completo: mede o intervalo entre a escrita do primeiro word na fila TX e a leitura completa da resposta da fila RX, tratando cada mensagem de forma síncrona (uma de cada vez). Os resultados obtidos apontam RTT médio de **10.228 ns**, com percentil 99 de **10.310 ns**. O pior caso observado foi de **1.146.360 ns**, o que eleva o jitter max–min para **1.136.220 ns**; esse valor, contudo, deve ser interpretado como cauda do sistema operacional ARM Linux e do caminho MMIO, não como variabilidade do núcleo FPGA. O núcleo C++ equivalente executa a mesma decisão em 315 ns no mesmo processador, resultando em software aproximadamente **32,4× mais rápido** em latência fim a fim na configuração atual.

Tabela 5 – Latência lógica vs. latência fim a fim na DE10-Nano

Medição	Descrição	Valor
Latência lógica FPGA	3 ciclos (on-chip), sem comunicação	60 ns
Latência lógica SW	Núcleo C++ equivalente no ARM Cortex-A9	315 ns
RTT ARM→FPGA→ARM	Escrita TX + decisão FPGA + leitura RX	10.228 ns médio; 10.310 ns p99
Speedup núcleo	FPGA lógica vs. C++ lógica (sem comun.)	5,253× mais rápido
Relação RTT vs. SW	Latência fim a fim vs. C++ equivalente	~32,4× mais lento

Fonte: Medições com `fpga\_benchmark -mode full` e `-mode fpga-sync` na DE10-Nano..

Esses resultados não invalidam a demonstração funcional nem a aceleração do núcleo lógico. Eles revelam que o gargalo desta arquitetura está principalmente na latência de ida e volta entre ARM e FPGA, e não na capacidade de vazão exigida pelo trabalho.

Suficiência de throughput no escopo do TCC. O modo `fpga-mmio` já faz envio em *pipeline*, mantendo múltiplas mensagens em voo simultaneamente nas filas circulares, e foi o responsável pela vazão de 102.381 msg/s. Como cada mensagem movimenta 64 bytes de payload útil entre evento e resposta, isso equivale a aproximadamente 6,55 MB/s; considerando cerca de 20 acessos MMIO de 32 bits por mensagem, o tráfego efetivo no barramento permanece por volta de 8,19 MB/s. Esse valor é uma fração pequena da capacidade bruta inferida de uma interface de 32 bits a 50 MHz, próxima de 200 MB/s. Em modo síncrono (`fpga-sync`), o ARM espera a resposta antes de enviar a próxima mensagem, o que limita a vazão teórica a aproximadamente **97.772 msg/s** quando se considera o RTT médio medido — valor coerente com os 102.381 msg/s obtidos no modo em *pipeline*. Assim, o MMIO não se mostrou insuficiente em throughput para as metas desta monografia; seu impacto principal concentrou-se em latência e ocupação do ARM.

Arquitetura de produção: FPGA sem ARM no caminho crítico. A limitação fundamental do protótipo não é o DMA nem o polling: é a presença do ARM Linux no laço crítico. Em sistemas profissionais de HFT, a FPGA é posicionada diretamente no caminho de dados da rede (Ethernet → FPGA → Ethernet), eliminando o sistema operacional da cadeia de latência (LEBER; GEIB; LITZ, 2011; LOCKWOOD *et al.*, 2012). O ARM (ou CPU equivalente) opera exclusivamente no *caminho frio*: configuração de parâmetros antes do pregão, monitoramento de posição, gestão de risco e envio de ordens de tamanho não urgente. Nessa topologia, a latência fim a fim cai para 100–500 ns porque os únicos componentes no laço crítico são a interface MAC, o *parser* de protocolo em FPGA e a lógica de decisão — todos síncronos e determinísticos.

6.7.5 Eficiência energética

Além da latência, a eficiência energética é uma vantagem estrutural de FPGAs em relação a CPUs de propósito geral. Um processador, seja ARM ou x86, dissipa potência em recursos que não participam da decisão de trading: unidade de controle de threads, hierarquia de cache, preditor de desvios e interfaces de memória geral. Na FPGA, apenas a lógica sintetizada para a tarefa específica comuta, e o consumo está diretamente vinculado à carga de trabalho do circuito.

Esta subseção não descreve um benchmark medido diretamente na DE10-Nano com amperímetro ou instrumento equivalente. Trata-se de um cálculo de energia por decisão a partir de potência típica e latência de processamento, usado para fornecer uma comparação de ordem de grandeza entre substratos computacionais.

A Tabela 6 apresenta estimativas de energia por decisão para as três classes de substrato. Os valores são estimativas de ordem de grandeza baseadas em potência típica dos dispositivos e na latência de processamento medida; não foram obtidos por instrumentação direta de corrente.

Tabela 6 – Estimativa de energia por decisão por substrato

Substrato	Potência típica	Latência lógica	Energia/decisão (est.)
FPGA Cyclone V (lógica)	~50 mW (dinâmica)	60 ns	~3 nJ
ARM Cortex-A9 (DE10-Nano)	~1,5 W (núcleo)	315 ns	~0,47 μ J
x86 moderno (ref. servidor)	~15 W (por núcleo)	~30 ns	~0,45 μ J

Fonte: Potências típicas extraídas dos respectivos *datasheets*; latência de x86 estimada por extrapolação da relação ARM/x86 para a mesma carga algorítmica. Valores são aproximados por uma ordem de grandeza..

A FPGA apresenta energia por decisão cerca de duas ordens de grandeza menor do que as CPUs de referência consideradas, mesmo sendo mais lenta em latência lógica do que um x86 moderno. Esse ganho energético é relevante em datacenters de co-location onde o custo de energia e resfriamento é proporcional à potência instalada (HASBROUCK; SAAR, 2013).

6.7.6 Comparação com trabalhos relacionados

Para uma comparação significativa, é necessário alinhar os escopos: este trabalho mede a latência fim a fim como RTT ARM→FPGA→ARM (**10.228 ns** médio, **10.310 ns** p99), dominado pela ponte MMIO. Os trabalhos de referência medem o pipeline completo incluindo interface de rede, obtendo latências sub-microsegundo por posicionarem a FPGA diretamente no caminho de dados sem sistema operacional no laço crítico (LEBER; GEIB; LITZ, 2011; LOCKWOOD *et al.*, 2012; BOUTROS *et al.*, 2017).

Leber, Geib e Litz (2011) demonstram que operações de Order Book e decisão em FPGA com pipeline de rede integrado atingem latência inferior a 1 μ s. Boutros *et al.* (2017) obtêm resultado comparável usando síntese de alto nível em C/C++, demonstrando que ferramentas HLS produzem circuitos competitivos com RTL manual — argumento análogo ao uso de MATLAB HDL Coder neste trabalho. No caso de Boutros *et al.* (2017), a latência de 269 ns do subsistema HFT implica uma taxa teórica de aproximadamente 3.717.472 mensagens por segundo, valor coerente com a classe de hardware empregada e com o posicionamento da FPGA diretamente no caminho de rede. Lockwood *et al.* (2012) focam na camada de rede e transporte com latência similar para o pipeline completo.

A Tabela 7 estende a comparação além dos trabalhos acadêmicos, incluindo referências de latência para sistemas de software profissional de HFT. Esses números evidenciam os diferentes pontos de operação ao longo do eixo de complexidade arquitetural versus desempenho.

A latência média de 10.228 ns deste protótipo é superior à de todos os sistemas FPGA de referência por uma razão estrutural única: a presença do ARM Linux no laço crítico. Mesmo assim, a vazão medida de 102.381 msg/s já supera a meta de 100.000 msg/s definida para o TCC. Portanto, a distância para plataformas profissionais não é um argumento contra FPGAs, nem uma evidência de insuficiência de MMIO para este protótipo; ela é, sobretudo, uma

Tabela 7 – Comparação de latência fim a fim entre arquiteturas de HFT

Referência	Latência total	Tecnologia	ARM/CPU no laço
Este trabalho	10.228 ns médio; 10.310 ns p99	Cyclone V SoC DE10-Nano, 50 MHz, MMIO via ARM	Sim (gargalo)
Leber, Geib e Litz (2011)	$<1 \mu s$	FPGA + rede inte- grada	Não
Boutros <i>et al.</i> (2017)	869 ns RTT; 269 ns subsistema HFT	Kintex UltraScale XCKU115, 156 MHz, HLS + rede integrada	Não
Lockwood <i>et al.</i> (2012)	$<1 \mu s$	FPGA, biblioteca rede + livro	Não
FPGA produção (co-location)	$\sim 100\text{--}500$ ns	FPGA direto Eth→FPGA→Eth	Não
<i>Kernel bypass</i> (DPDK)	$\sim 1\text{--}3 \mu s$	Software, bypass do SO	Sim
Software padrão (Linux)	$\sim 50\text{--}500 \mu s$	Pilha de rede do ker- nel	Sim
ASIC dedicado	$\sim 10\text{--}50$ ns	Hardware customi- zado por protocolo	N/A

Fonte: Dados de trabalhos acadêmicos extraídos das publicações originais. Latências de *kernel bypass* e software padrão são estimativas consolidadas da literatura de sistemas (RIZZO, 2012; JEONG *et al.*, 2014). Latências de FPGA de produção e ASIC são valores reportados por fabricantes e operadores de co-location; não estão disponíveis em literatura acadêmica aberta. N/A = não aplicável..

consequência da escolha arquitetural de manter o ARM no caminho crítico em uma plataforma acadêmica de baixo custo.

Há ainda um ponto econômico importante. A comparação com Boutros *et al.* (2017) envolve classes de hardware muito diferentes. Enquanto a DE10-Nano utilizada neste TCC custa aproximadamente US\$190 em condição acadêmica (Terasic Technologies, 2026), o FPGA XCKU115-2FLVA1517E empregado naquele trabalho aparece listado por US\$10.290,57 (Mouser Electronics, 2026). Assim, o componente de alto desempenho sozinho custa mais de 54 vezes o valor da plataforma inteira usada aqui. Essa diferença de custo, somada à diferença de frequência (156 MHz contra 50 MHz) e à ausência de rede direta em nossa placa, ajuda a contextualizar por que um throughput da ordem de 3,7 milhões de mensagens por segundo em um sistema profissional não pode ser exigido de um protótipo acadêmico baseado em DE10-Nano sem considerar o desenho de comunicação.

Vale notar que sistemas de software com *kernel bypass* (DPDK, OpenOnload) alcançam latências de $1\text{--}3 \mu s$ ao eliminar o kernel do caminho de rede, mas ainda sofrem com jitter por executarem em CPUs de propósito geral com interrupções, gerenciamento de energia e contenção entre *cores* (RIZZO, 2012). Em contraste, um pipeline FPGA sem ARM atinge latência similar ou inferior *com jitter próximo de zero*, pois o único caminho não síncrono — a rede física — tem variação dominada pelo hardware de PHY, não pelo processamento. Na extremidade

oposta do espectro, ASICs dedicados (como os usados por algumas firmas de HFT de alta capitalização) atingem 10–50 ns de latência total, mas exigem ciclos de design de 12–24 meses e custo de *tape-out* na ordem de milhões de dólares, tornando-os inviáveis fora de operações de escala industrial.

7 Conclusão

Este trabalho apresentou o desenvolvimento e a documentação de uma plataforma de HFT de baixo custo baseada em co-projeto ARM–FPGA, com foco principal na criação de estratégias de trading em MATLAB e geração de hardware por meio do MATLAB HDL Coder. A infraestrutura desenvolvida cobre o caminho necessário para alimentar uma estratégia gerada por HLS: dados sintéticos de mercado, normalização no ARM, comunicação com a FPGA, Order Book simplificado, decisão em hardware, validação por testes e preparação para execução na DE10-Nano.

O objetivo geral foi atendido no nível funcional: o sistema permite isolar a estratégia como função MATLAB, gerar VHDL com HDL Coder, integrar o bloco gerado ao motor de decisão e executar o caminho de dados entre software e FPGA. O trabalho também consolidou o contrato entre C++ e VHDL, enfatizando registradores, ponteiros, capacidade efetiva de filas, cálculo dinâmico da base RX, largura de oito palavras por quadro e ordem de empacotamento. Esse contrato é uma contribuição de engenharia essencial, pois mantém estável a infraestrutura sobre a qual novas estratégias podem ser testadas.

Entre as vantagens observadas, destacam-se a modularidade da arquitetura, a separação adequada entre software flexível e hardware determinístico, a possibilidade de alterar a regra de estratégia em MATLAB e regenerá-la por HDL Coder, e a existência de testes em diferentes níveis. A abordagem por filas circulares é simples, dispensa bloqueios entre produtor e consumidor e facilita a depuração. A modelagem do Order Book por profundidade fixa favorece síntese em FPGA e evita estruturas dinâmicas incompatíveis com RTL.

Os objetivos específicos também foram contemplados. A plataforma foi organizada para que a estratégia seja o principal bloco substituível pelo usuário. O estudo teórico apresentou os conceitos necessários de HFT, Order Book, FPGA, HLS, MATLAB HDL Coder, FAST, MMIO, Avalon-MM e métricas de desempenho. A documentação do contrato ARM–FPGA foi incorporada ao texto e detalhada em apêndice. Os módulos de software, hardware e infraestrutura foram descritos com suas funções, vantagens, limitações e integração. Por fim, a validação funcional foi organizada em uma matriz que relaciona testes a riscos técnicos.

Do ponto de vista acadêmico, a contribuição central não é uma estratégia de negociação lucrativa, mas um fluxo de criação de estratégias para FPGA. Essa distinção é importante. Estratégias de HFT reais dependem de dados proprietários, controle de risco, colocação de ordens, posição, custos e conformidade. A plataforma, por sua vez, demonstra como uma aplicação sensível a latência pode ser decomposta em software e hardware, como representar eventos em ponto fixo, como preservar ordenação em uma fila circular e como usar HLS para reduzir o esforço de implementação do bloco decisório.

As limitações também são claras. O feed é sintético e usa TCP, não dados reais de bolsa. O Order Book é por nível agregado, não por ordem individual. A estratégia é didática e não considera posição, risco, custos, execução ou conformidade regulatória. A campanha

quantitativa de latência, vazão e aceleração já foi realizada. Além disso, MMIO com polling é suficiente para o protótipo desenvolvido, mas não representa a arquitetura de menor latência possível.

Outra limitação é o escopo das métricas disponíveis. A execução em placa já confirma quantitativamente o desempenho do núcleo lógico e da ponte MMIO, mas a medição empírica de energia ainda não foi realizada. Essa cautela evita transformar uma caracterização parcial de desempenho em afirmação absoluta sobre eficiência global da plataforma.

Os resultados quantitativos obtidos com o benchmark revelam uma distinção importante entre aceleração de núcleo lógico e latência fim a fim. A FPGA executa o núcleo de livro e decisão em 60 ns com jitter zero, vazão de 102.381 mensagens por segundo e aceleração de $5,253\times$ em relação ao núcleo C++ equivalente. Entretanto, o custo de comunicação pela ponte MMIO eleva o RTT ARM→FPGA→ARM para 10.228 ns em média, com p99 de 10.310 ns e cauda longa associada ao sistema operacional ARM Linux. Esse resultado mostra que o gargalo atual não está na lógica de hardware, mas na interface de comunicação. Sistemas de produção resolvem esse problema posicionando a FPGA diretamente no caminho de rede, mas isso ultrapassa o escopo deste protótipo.

Essa conclusão precisa ser lida também sob a ótica de custo e acessibilidade. O protótipo foi construído sobre uma DE10-Nano adquirida em condição acadêmica por aproximadamente US\$190 (Terasic Technologies, 2026). Em contraste, Boutros *et al.* (2017) utilizam uma FPGA Kintex UltraScale XCKU115 operando a 156 MHz; o componente XCKU115-2FLVA1517E aparece listado por US\$10.290,57 (Mouser Electronics, 2026), isto é, mais de 54 vezes o valor da plataforma completa adotada neste trabalho. Assim, a contribuição deste TCC não é competir em throughput absoluto com plataformas profissionais, mas demonstrar que uma infraestrutura funcional de HFT acadêmico pode ser construída com orçamento muito inferior, mantendo ainda assim determinismo interno e aceleração real no núcleo lógico.

Como trabalhos futuros, recomenda-se:

- **Medição de energia por decisão:** instrumentar o consumo de corrente do Cyclone V com medição direta (amperímetro no trilho de alimentação) durante o benchmark, obtendo energia por decisão empírica em vez da estimativa por ordem de grandeza apresentada na Tabela 6.
- **Estratégias mais complexas em MATLAB:** avaliar regras com médias móveis, bandas de Bollinger ou correlação entre símbolos, medindo o impacto no uso de recursos e na latência lógica à medida que a complexidade do código MATLAB aumenta.
- **Modelo de eventos enriquecido:** ampliar para cancelamentos, execuções parciais e múltiplos níveis de profundidade, validando a robustez da infraestrutura frente a padrões de livro mais próximos do real.

- **Controle de risco e posição:** adicionar limites de posição, janela de perda máxima e conformidade de ordens como módulo separado, mantendo a estratégia MATLAB como bloco isolado da lógica de risco.

A evolução natural do projeto pode ser dividida em três frentes. A primeira é experimental: coletar métricas com instrumentação adequada, especialmente medição de energia. A segunda é algorítmica: ampliar o conjunto de estratégias geradas por MATLAB e observar como a complexidade afeta o núcleo lógico. A terceira é funcional: enriquecer o modelo de eventos, incluir controle de posição, limites de risco e custos de transação. Essas frentes são complementares e devem ser conduzidas nessa ordem para evitar que uma estratégia mais complexa esconda problemas básicos de infraestrutura.

Conclui-se que a proposta de uma plataforma de HFT de baixo custo orientada à geração de estratégias por MATLAB HDL Coder é viável no nível funcional apresentado. Mesmo sem atingir o escopo de um sistema financeiro real, o protótipo demonstra que uma estratégia escrita em MATLAB pode ser tratada como bloco substituível, gerada para VHDL por HLS e integrada a uma infraestrutura ARM–FPGA com Order Book, filas MMIO e validação cruzada. Esses elementos constituem uma base sólida para a continuidade do projeto, especialmente na coleta de métricas quantitativas e na avaliação de novas estratégias descritas em MATLAB.

Referências

- ALDRIDGE, I. **High-Frequency Trading: A Practical Guide to Algorithmic Strategies and Trading Systems**. 2. ed. [S.l.]: Wiley, 2013.
- BOUTROS, A. *et al.* Build fast, trade fast: Fpga-based high-frequency trading using high-level synthesis. *In: 2017 International Conference on ReConFigurable Computing and FPGAs*. Cancun, Mexico: IEEE, 2017. p. 1–6.
- BROGAARD, J.; HENDERSHOTT, T.; RIORDAN, R. High-frequency trading and price discovery. **The Review of Financial Studies**, v. 27, n. 8, p. 2267–2306, 2014.
- CONT, R.; STOIKOV, S.; TALREJA, R. A stochastic model for order book dynamics. **Operations Research**, v. 58, n. 3, p. 549–563, 2010.
- FIX Trading Community. **FIX Adapted for STreaming Technical Standard**. [S.l.], 2006. Disponível em: <https://www.fixtrading.org/standards/fast-online/>. Acesso em: 13 abr. 2026.
- GOULD, M. D. *et al.* Limit order books. **Quantitative Finance**, v. 13, n. 11, p. 1709–1742, 2013.
- HARRIS, D. M.; HARRIS, S. L. **Digital Design and Computer Architecture**. 2. ed. [S.l.]: Morgan Kaufmann, 2013.
- HASBROUCK, J.; SAAR, G. Low-latency trading. **Journal of Financial Markets**, v. 16, n. 4, p. 646–679, 2013.
- HENDERSHOTT, T.; JONES, C. M.; MENKVELD, A. J. Does algorithmic trading improve liquidity? **The Journal of Finance**, v. 66, n. 1, p. 1–33, 2011.
- Intel Corporation. **Cyclone V and Arria V SoC Device Design Guidelines (AN 796)**. [S.l.], 2022. Disponível em: <https://www.intel.com/content/www/us/en/docs/programmable/683360/18-0/lightweight-hps-to-fpga-bridge.html>. Acesso em: 12 maio 2026.
- Intel Corporation. **Avalon Interface Specifications**. [S.l.], 2024. Disponível em: <https://www.intel.com/content/www/us/en/docs/programmable/683091/current/avalon-memory-mapped-interfaces.html>. Acesso em: 13 abr. 2026.
- Intel Corporation. **Intel Quartus Prime Platform Designer User Guide**. [S.l.], 2024. Disponível em: <https://www.intel.com/content/www/us/en/docs/programmable/683609/current/platform-designer-system-integration-tool.html>. Acesso em: 13 abr. 2026.
- Intel Corporation. **Cyclone V FPGA and SoC FPGA Product Brief**. [S.l.], 2025. Disponível em: <https://cdrdv2-public.intel.com/853455/cyclone-v-fpgas-and-socs-product-brief.pdf>. Acesso em: 13 abr. 2026.
- JEONG, E. *et al.* mTCP: A highly scalable user-level TCP stack for multicore systems. *In: Proceedings of the 11th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. [S.l.]: USENIX Association, 2014. p. 489–502.
- LEBER, C.; GEIB, B.; LITZ, H. High frequency trading acceleration using fpgas. *In: 2011 21st International Conference on Field Programmable Logic and Applications*. [S.l.: s.n.], 2011. p. 317–322.

LOCKWOOD, J. W. *et al.* A low-latency library in fpga hardware for high-frequency trading. *In: 2012 IEEE 20th Annual Symposium on High-Performance Interconnects*. [S.l.: s.n.], 2012. p. 9–16.

Mouser Electronics. **AMD / Xilinx XCKU115-2FLVA1517E Product Page**. [S.l.], 2026. Disponível em: <https://br.mouser.com/ProductDetail/AMD-Xilinx/XCKU115-2FLVA1517E?qs=rrS6PyfT74d8eGSliGMnXA%3D%3D>. Acesso em: 12 maio 2026.

Object Computing, Inc. **mFAST: a C++ FIX/FAST Encoder and Decoder**. [S.l.], 2026. Disponível em: <https://github.com/objectcomputing/mFAST>. Acesso em: 13 abr. 2026.

PATTERSON, D. A.; HENNESSY, J. L. **Computer Organization and Design: The Hardware/Software Interface**. 5. ed. [S.l.]: Morgan Kaufmann, 2017.

RIZZO, L. netmap: A novel framework for fast packet i/o. *In: Proceedings of the USENIX Annual Technical Conference (ATC)*. [S.l.]: USENIX Association, 2012. p. 101–112.

Terasic Technologies. **DE10-Nano User Manual**. [S.l.], 2024. Disponível em: <https://www.terasic.com.tw/cgi-bin/page/archive.pl?Language=English&CategoryNo=167&No=1046>. Acesso em: 13 abr. 2026.

Terasic Technologies. **DE10-Nano Development and Education Board – Order Page**. [S.l.], 2026. Disponível em: <https://www.terasic.com.tw/cgi-bin/page/archive.pl?CategoryNo=167&Language=English&No=1046>. Acesso em: 12 maio 2026.

The MathWorks, Inc. **Fixed-Point Designer Documentation**. [S.l.], 2026. Disponível em: <https://www.mathworks.com/help/fixedpoint/>. Acesso em: 13 abr. 2026.

The MathWorks, Inc. **HDL Coder Documentation**. [S.l.], 2026. Disponível em: <https://www.mathworks.com/help/hdlcoder/>. Acesso em: 13 abr. 2026.

APÊNDICE A – Contrato de Comunicação ARM–FPGA

Este apêndice consolida o contrato de comunicação utilizado pelo protótipo. Ele complementa a descrição do Capítulo 4 e deve ser mantido alinhado aos arquivos `cpp/src/fpga_shared_stream.h`, `cpp/src/fast_receiver.cpp`, `vhdl/arm_fpga_shared_stream_bridge.vhd` e `docs/arm-fpga-shared-memory-stream`.

A.1 Registradores

Tabela 8 – Registradores da interface MMIO

Offset	Nome	Acesso	Dono	Descrição
0x000	MAGIC	RO	FPGA	Valor 0x48465431.
0x004	VERSION	RO	FPGA	Versão do protocolo.
0x008	CTRL	RW	ARM	Bit 0 reinicia ponteiros.
0x00C	STATUS	RO	FPGA	Bits de disponibilidade das filas.
0x010	TX_HEAD	RW	ARM	Publicação de comandos.
0x014	TX_TAIL	RO	FPGA	Consumo de comandos.
0x018	RX_HEAD	RO	FPGA	Publicação de respostas.
0x01C	RX_TAIL	RW	ARM	Consumo de respostas.
0x020	TX_DEPTH	RO	FPGA	Profundidade da fila TX.
0x024	RX_DEPTH	RO	FPGA	Profundidade da fila RX.
0x028	SLOT_WORDS	RO	FPGA	Palavras por quadro.

Fonte: Autoria própria (2026).

A.2 Regras de filas

As filas TX e RX usam a mesma semântica:

- vazia: `head == tail`;
- cheia: `next(head) == tail`;
- capacidade útil: `DEPTH - 1`;
- produtor escreve payload antes de publicar `head`;
- consumidor lê payload antes de publicar `tail`.

Para `DEPTH = 64` e `SLOT_WORDS = 8`, a base TX é 0x100, cada slot tem 32 bytes e a base RX é 0x900.

A.3 Formato de evento

O evento enviado do ARM para o FPGA usa oito palavras. A palavra de sequência é preservada para correlação entre entrada e saída. O identificador de símbolo é numérico para

evitar processamento textual em hardware. O preço é escalado por 10^4 para representar casas decimais com inteiros.

Tabela 9 – Formato do quadro ARM–FPGA

Palavra	Campo	Semântica
word0	Sequência	Número de sequência da mensagem sintética.
word1	Símbolo	Identificador numérico atribuído pelo receptor.
word2	Preço	Preço em escala 10^4 .
word3	Quantidade	Quantidade agregada no nível de preço.
word4	Tipo de evento	Atualização/inserção, remoção ou reset.
word5	Lado	Compra ou venda.
word6	Reserva	Campo reservado para extensão futura.
word7	Reserva	Campo reservado para extensão futura.

Fonte: Autoria própria (2026).

A.4 Formato de resposta

O software interpreta a resposta do FPGA como:

- word0: sequência;
- word1: ação;
- word2: melhor compra em escala 10^4 ;
- word3: quantidade na melhor compra;
- word4: melhor venda em escala 10^4 ;
- word5: quantidade na melhor venda;
- word6: *spread*;
- word7: desbalanceamento assinado.

A.5 Checklist de alteração do contrato

Quando o contrato ARM–FPGA for alterado, os seguintes itens devem ser revisados em conjunto:

- mapa de registradores no VHDL e na documentação;
- largura `G_SLOT_WORDS` no VHDL e `kFrameWords` no C++;
- cálculo da base RX no software, nos testbenches e no texto;

- ordem das palavras em `fast_receiver.cpp`;
- decodificação das palavras em `order_book_core.vhd`;
- montagem da resposta em `generated_strategy_core.vhd`;
- testes C++ e VHDL que validam fila cheia, *wrap-around* e resposta;
- comandos e exemplos de execução no README e neste apêndice.

Esse checklist existe porque mudanças pequenas em interfaces binárias podem causar erros difíceis de diagnosticar. Em particular, uma alteração de largura de slot muda a base RX; uma alteração de ordem de palavra pode fazer preço ser interpretado como quantidade; e uma alteração de evento sem atualização do livro pode gerar snapshots inconsistentes.

APÊNDICE B – Comandos de Construção, Teste e Execução

Este apêndice reúne comandos úteis para reproduzir a construção e validação do protótipo.

B.1 Validação local

Listagem 2 – Comandos de teste local

```
1 make docker-image
2 make cpp-test
3 make cpp-smoke
4 make vhdl-test-all
```

Fonte: Autoria própria (2026).

O alvo `cpp-test` valida o wrapper MMIO com arquivo temporário. O alvo `cpp-smoke` executa o gerador e o receptor em modo software. O alvo `vhdl-test-all` executa testbenches da ponte, livro, estratégia, motor completo e wrapper Avalon-MM.

B.2 Geração da estratégia

Listagem 3 – Geração do bloco de estratégia por HDL Coder

```
1 make matlab-image
2 make matlab-hdl-generate
```

Fonte: Autoria própria (2026).

O primeiro comando prepara a imagem esperada para execução do MATLAB em ambiente controlado. O segundo executa o script `matlab/hdl_generator.m`, que chama a geração VHDL da função `strategy.m`. Após essa etapa, recomenda-se executar o testbench da estratégia e o testbench do motor completo, pois uma mudança no bloco gerado pode alterar latência, portas ou comportamento.

B.3 Geração HDL e build para DE10-Nano

Após a programação do FPGA, o feed e o receptor podem ser executados na placa:

O endereço `0xFF200000` corresponde à base típica da ponte leve HPS-FPGA na DE10-Nano quando o componente está no offset zero do Platform Designer. Se o componente for posicionado em outro offset, a base deve ser ajustada.

Listagem 4 – Fluxo previsto para DE10-Nano

```
1 make de10-sysroot
2 make matlab-login
3 make build
4 make quartus-program
5 make de10-enable-bridges
6 make deploy
```

Fonte: Autoria própria (2026).

Listagem 5 – Execução na placa

```
1 ssh root@192.168.7.1 'cd /home/root && ./fast_data_feed '
2 ssh root@192.168.7.1 \
3 'cd /home/root && HFT_FPGA_MMIO_BASE=0xFF200000 ./fast_receiver '
```

Fonte: Autoria própria (2026).

Tabela 10 – Variáveis de ambiente do receptor

Variável	Função
HFT_FPGA_MMIO_BASE	Base física do periférico. Quando ausente, o receptor opera em modo software-only.
HFT_FPGA_MMIO_SPAN	Tamanho da região mapeada. Quando ausente, usa valor padrão suficiente para a geometria esperada.
HFT_FPGA_MMIO_DEV	Dispositivo usado para mapeamento MMIO. Por padrão, /dev/mem.

Fonte: Autoria própria (2026).

B.4 Variáveis de ambiente

A presença de `HFT_FPGA_MMIO_BASE` muda o modo de operação do receptor. Por isso, testes de software devem ser executados sem essa variável, enquanto testes em placa devem defini-la explicitamente. Em ambientes compartilhados, recomenda-se imprimir a base efetiva antes de enviar eventos para evitar confusão entre offset de Platform Designer e base da ponte leve HPS-FPGA.

B.5 Sequência recomendada de depuração

Uma sequência prática de depuração é:

1. executar `make cpp-test` para validar a semântica de filas no wrapper;
2. executar `make cpp-smoke` para validar FAST, TCP e decodificação sem FPGA;
3. executar `make vhdl-test-all` para validar os blocos de hardware em simulação;
4. gerar e programar o bitstream;

5. habilitar pontes HPS–FPGA;
6. executar o receptor com `HFT_FPGA_MMIO_BASE` e verificar leitura de `MAGIC`;
7. iniciar o feed e observar respostas [FPGA->ARM] .

Se a leitura de `MAGIC` falhar, o problema provavelmente está em endereço, bitstream ou ponte HPS–FPGA. Se `MAGIC` estiver correto, mas `Send` falhar por fila cheia, deve-se investigar consumo TX no FPGA, reset e handshakes internos. Se respostas não aparecem, mas TX é consumida, o foco passa a ser *Order Book*, estratégia e fila RX.